Spatial- temporal analysis of signals to detect drift of air quality sensors

CARRENO Diego[1], MEJIA Johan[2], MORENO Tatiana[3], and AMAL Ilias[4]

[1,2,3]MSc. IT: Data Science
[4]Diplôme d'Ingénieur
IMT ATLANTIQUE

March 17, 2021

1 Introduction

Nowadays, a great amount of information is available in different science domains such as: marketing, mobility and traffic parameters, sales, finance and banking data, cellular networks and even information about our health from health devices and application. This information can be very useful if it is analyzed correctly, behavioral patterns can be extracted and help improve processes, for example, identify tastes in movies or user series to find patterns and improve a user's experience on a digital platform such as Netflix or amazon, analyze a person's neural activity to identify degenerative diseases in the brain, or determine factors that influence the increase of accidents in different parts of a city to improve transportation in a city, among many other applications. That is why data analysis and processing is a field that is gaining importance in our current society. However, the complexity of this type of applications where there is a need to find relationships between many agents represents a challenge and requires the need for new, non-standard analysis techniques.

Graphs offer the ability to model data and capture complex interactions between the different features. An approach to graphical signal processing theory can be found in [1, 2], with its relation to classical digital signal processing, and the recent advances made in the domain of the different algorithms and a list of possible applications. There are a great number of examples of how graphs can be used to extract information about the behaviour of sensor networks [3], determine exposure to building contamination [4] or model the behaviour of the human brain [1].

A graph is composed by two principal components: vertices (also called nodes) which are connected by edges. The Figure 1 shows examples of network signals. Time series, are represented by directed graphs as in Figure 1a, in the case of periodic signals this representation is made in a cycle ,i.e. the last vertex is connected to the first one. Each node corresponds to a time instant; all edges are direct (only one direction possible) and have a weight of 1, reflecting the similarity between intervals. The Figure 1b represents an example of a network in which each vertice can represent different things like: buildings, neurons, pixels, sensors, etc. and the edges represent the relationships between two points on edge can correspond to distances, similarities in measurements, etc. The Figure 1c represents the measurements taken by a network of sensors at a given time instant.

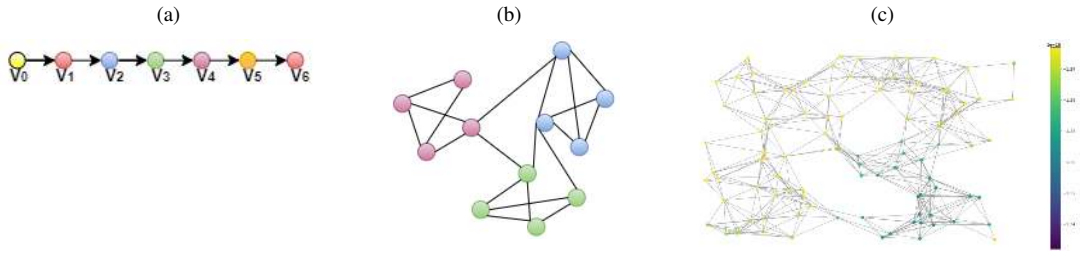


Figure 1: Examples of graph signals. Signal values are represented with different colors. (a) Time series graph representation. (b) Example of graph. (c) Signal graph representation

The study of sensor network is one of the most natural applications of Graph Signal Processing (GSP) methodology. A sensor network allows to follow a complex phenomenon by observing the temporal evolution of the values at various points. This allows, to carry out a meteorological or seismic monitoring, detecting a cloud or an earthquake. However, the sensors used are subject to failure due to wear and tear or possible interference, which makes certain observed values irrelevant. Therefore it is important to determine where (sensor) and when there is a failure in the network. To solve this task we will used the Graph Fourier Transform (GFT).

The GFT is a very important tool in the analysis of graphs signal. In an analogous manner to the Discrete Fourier Transform (DFT), using GFT one may examine graph signals in the graph frequency domain, and, for instance, remove noise by attenuating

high graph-frequencies. GFT has also lead to significant new insights in problems such as smoothing and denoising, segmentation, sampling and classification of graph data [5]. Since the objective of this work is to analyze the behavior of the signals measured in a sensor network during a period of two months, it is required to use temporal graph signals, using a Joint time-vertex Fourier Transform (JFT) which give a joint variation of a signal in time and space.

In general a spectrogram is a visual representation of the strength of a signal over time at various frequencies. In other words, it allows to observe when the signal presents a greater variability; analogously in the graph signals it allows to observe when a signal in a network presents high frequencies, that is when several sensors present very different measurements between them or when a vertex in time presents many differences in its measurements. This representation allows to identify and to analyze clusters and abnormal behavior. The following can be very helpful for the project's goal of detecting drift in the sensor network.

The Figure 2 shows a spectrogram example on a path graph [6]. Figure 2a A signal on a path graph that is composed of three different signal for three different segment of a path graph. Figure 2b Spectrogram of the signal. The vertex indices are on the horizontal axis, and the frequency indices are on the vertical axis. The spectrogram gives us information on how these sensors could be grouped without the need to analyze in detail the signal a-priori.

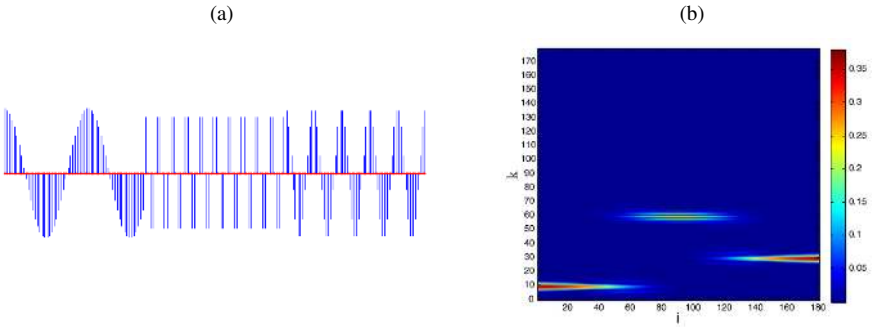


Figure 2: Spectrogram example on a path graph; (a) A signal on the path graph that is composed of three different signals for three different segments of a path graph. (b) Spectrogram of the signal. Taken from [6].

The report has the following line: In Section 2 the methodology followed in order to detect the failure of the sensor network in the city of Pune is presented. Section 4 presents a theoretical explanation of the graphs, the GFT, JFT and spectrograms. In Section 6 we fast derive an exploration of the data that compose the network, and we also present the different methods used in order to complete missing information, in order to build the graph based on real information on the location of the sensors in Section 8. Below, we show the signal application measured by the sensors in the graph from the mathematical tools (Section 9), and the summary on the management of the project and distribution of works (Section 3). Finally some conclusions and future work are presented in Section 10.

2 Methodology

A sensor is a device capable of measuring changes in its environment, currently there are all kinds of sensors, capable of measuring everything from weather changes to air quality. However, due to their physical characteristics the sensors are prone to drift, either by prolonged use or by being exposed to difficult environments, humidity, high temperatures, etc. Drift is a low frequency change in the sensor value over time. This event considerably reduces the accuracy of the measurement which makes the study of phenomena difficult. Therefore the importance of detecting and correcting drift [7] in a sensor. In this paper we concentrate on the development of a method for improve the drift detection.

There are several methods in the literature to detect drift, however the majority of these methods are based on the signal history measured by each sensor independently and from this information detect abnormal signal behaviour at a specific time [8]. This work aims to include the location of each sensor in the network, and improve the drift detection.

Let $\mathcal{G}$ and $\mathcal{L}$ be the graphs that represent the spatial and temporal distribution of the sensor network, respectively, and $\mathbf{X}$ the matrix that represents the signals in the vertices v through the instants of time t that act on the graph $\mathcal{G}$ and $\mathcal{L}$, it is possible to estimate the Fourier transform of $\mathbf{X}$ acting on graphs $\mathcal{G}$ and $\mathcal{L}$ simultaneously by means of graph set $\mathcal{J}$, to have a notion of variability of the signals in space (difference of measurements between neighboring sensors) and in time (detecting abrupt changes in the signal from each sensor). From the estimation of the distance between two windows of the joint spectrogram, centered on all possible combinations of space and time (v_i, t_j with $i = 1, 2, 3, \cdots, V$ number of vertices and $j = 1, 2, 3, \cdots, T$ instants in time), it is possible to analyze the abrupt changes of the signal, with the purpose of detecting the instant and the group of vertices that had drift concept problems.

3 Project management

3.1 Importance of management

Project management is important because it ensures what is being delivered, is right, and will deliver real value against the business opportunity and it brings leadership and direction to projects. In addition it ensures there's a proper plan for executing on strategic goals and proper expectations are set around what can be delivered, by when, and for how much.

3.1.1 Data Management

Through data of air quality taken by numerous sensors spread in the city of Pune in India between November 2018 and October 2019, we analyse then prepare the data in a way we could achieve our goal of detect a drifting sensor, thus our data should have the five following: accuracy, completeness, reliability, relevance, and timeliness that correspond to data quality. We first analyse the behaviour of the different features that correspond to gas or index related to air quality to identify the ones that we could use in term of relevance and accuracy. Then we prepare the data by homogenise dates and filling the NaN values to obtain quality data in term of reliability and timeliness.

3.1.2 Code development

Using the python programming language and several libraries, we develop the approach to determine a failure in the sensor network, using spectrograms that summarize the behavior of the network in space and time with the `JFT`. There are two main tasks to obtain a true analysis about the real network: the creation of a network based on the real information about the location of the sensors in the city of Section 8. and the development of a spectrograph in Section 9.

In order to test the algorithm, it is implemented with a simulated signal in which the time and the damaged sensor is known in advance Sub-section 9.1. This algorithm is then applied to the real signals of the sensor network once the cleaning process is completed .

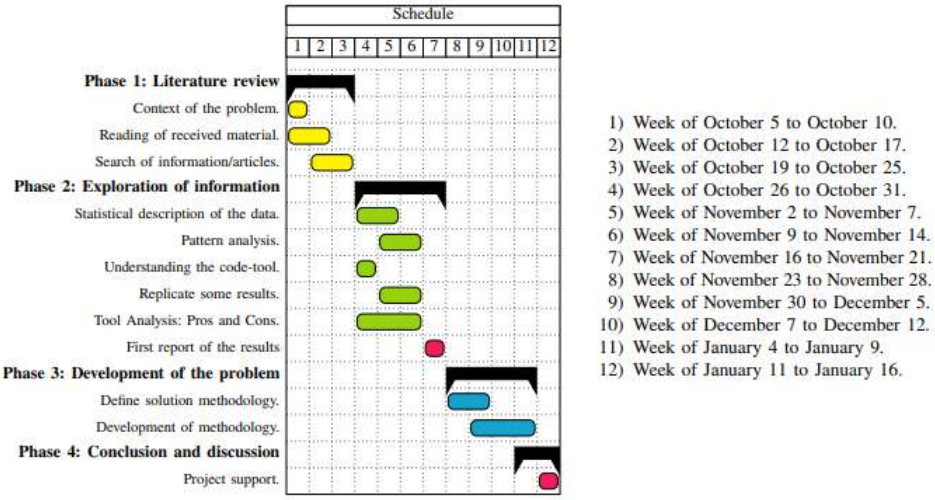


Figure 3: First planning made at the start of the project

At the beginning, we met in a room at IMT Atlantique to organise us. Indeed, we start with a first planning using the different elements of the project we were informed of at this moment, so without a total comprehension on the subject. This first planning was a way to identified key date like the date of the first and final report or presentation. According to the presentation of the project, we clearly divided our project into two parts, a code part to implement a model to detect beforehand drifting sensors and a data part were we are dealing with a big dataset to put it in our model. Each part is divided in sub tasks that consists at first to understand globally how the dataset and the model are constructed, then we explore them to find a way to respond to our problematic and finally try this method. So during our first meeting, we also assigned a part to each member: data or code. Then for each part , we try to identify what were the main way to work to respond to our objective that is to identify beforehand when a sensor will drift.

Our main first common task was to establish a State of Art in order that each member of the group understand well the different notion that will be implied in this project. Thenceforth, we start reading article using the same mathematical notion and other one dealing with a goal close to our. That part represented the first three weeks of our project, each week the meet to share what we understood of the different notion to be sure that everyone had a global vision to be able to work efficiently in the future. We also defined our role in the management project.

In parallel, we work also on the management aspect of this project to determinate the potential risk we could face during this project, show in Figure 4.

(a)

Défaillance et maîtrise des outils	
Classe : Risques techniques	**Probabilité** : Forte **Gravité** : Moyenne **Sévérité globale** : Acceptable
Causes potentielles ou réelles : - Performance des équipements de travail - Réseau non stable - Méconnaissance des méthodes à appliquer - Mauvaise compréhension de la théorie	
Impacts possibles : - Retard de livraison - Découragement de l'équipe	
Actions de réduction : - Demander des ressources, partager ses résultats - Respecter une méthodologie scientifique	

(b)

Respect des délais	
Classe : Risques de délais	**Probabilité** : Faible **Gravité** : Forte **Sévérité globale** : Acceptable
Causes potentielles ou réelles : - Complexité du projet - Planification non réaliste (estimer la durée tâche inconnu) - Réalisation de tâche non essentiel - Organisation de l'équipe	
Impacts possibles : - Objectif non atteint totalement au final - Trop grande perte de temps sur certaines tâches	
Actions de réduction : - Solliciter les encadrants (questions), partage des résultats - Respecter une méthodologie scientifique	

(c)

Manque de cohésion de l'équipe	
Classe : Risques humains	**Probabilité** : Faible **Gravité** : Forte **Sévérité globale** : Acceptable
Causes potentielles ou réelles : - Communication dans une langue étrangère - Dispersion géographique et dans d'autres projets - Absence et absence d'information sur le travail de ses camarades - Manque de réunion	
Impacts possibles : - Méconnaissance de la bonne avancé du projet - Incohérence des livrables - Charge de travail inégale	
Actions de réduction : - S'assurer de la bonne compréhension pendant les réunions - Rapport d'avancé individuelle	

Figure 4: Project risks evaluated in three phases: (a) tool failure, (b) time delays and (c) miscommunication.

After this first task, each group, the one responsible of the data and the other of the code start to make a first analysis. The code group studied a ancient code made by the professor to acquire knowledge and have a base that they work on. The data group analyse a dataset they get on air quality in Pune (India).

At this point, we had to change our organisation due to the global pandemic and the measure of confinement. Before this we could meet with our supervisors to share our progress and explain it in a way so that everyone can follow. Then we decide to communicate between us through Whatsapp and with the professor through Mattermost to ask question if needed or to have a feedback on that we working on. We also organise a meeting between us every Monday, so everyone is aware of the progress of each member, plus we can also discuss some point to lead the project at its end. In addition to this meeting, we had another one on Thursday with our supervisors to talk about our progress, our difficulties and see if the project is going in the wright direction. We prepare this meeting through a progress report to write what we did this week, our results and some issues we encounter, in addition every member could aware of the progress and the delay of the project. We also use Github and Google Colab to share and organise our work, Google Drive to prepare our presentation and Overleaf to write our report.

During our first meeting in this circumstance, we discuss about our first analysis on both part and explain how we think we will proceed. After that, we defined more clearly the different tasks we have to complete to achieve our goal, the new distribution of task is shown in Figure 3.1.2.

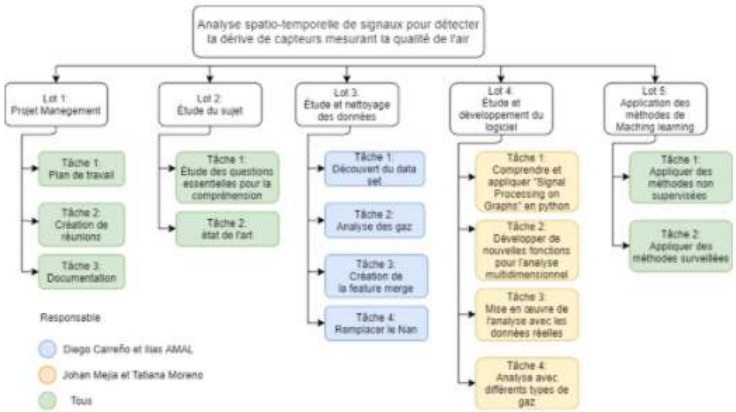
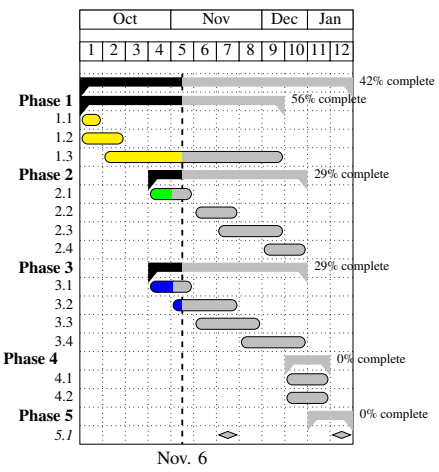


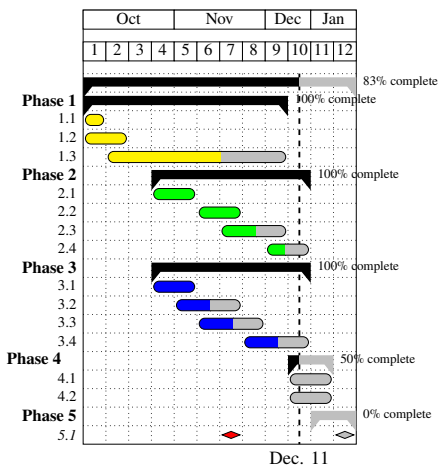
Figure 5: Task distribution list based on the sections planned for the project.

A week after that, we made a new planning with the new appropriate task we just defined and established deadlines to visualise if we progress well (Figure 6a). Since that moment, we progress well in the different tasks and respect our scheduling except of minor point that we need to add that extend the deadline of some tasks. We made a mid planning two weeks before the date of final report to focus on specific tasks because the deadline of the report was approaching and therefor needed to prioritised the tasks in case it was not possible to finish all the tasks we planned (Figure 6b)

(a)



(b)



(c)

Phase 1: State of the art	
1.1	Problem context
1.2	Reading of materials and documents received
1.3	Search for information/articles
Phase 2: Study and data cleaning	
2.1	Discovery of the dataset and some statistics
2.2	Gas analysis and models
2.3	Creation of the merge feature
2.4	Proposal to replace empty values
Phase 3: Code Review and Implementation	
3.1	Understanding and applying GSP in Python
3.2	Develop functions for spatio-temporal analysis
3.3	Implementation with real data
3.4	Multidimensional analysis
Phase 4: Maching Learning Methods	
4.1	Application of unsupervised methods
4.2	Application of supervised methods
Phase 5: Conclusions and Final Report	
5.1	Report submission dates.

Figure 6: (a) First planning made at the start of the project, (b) Evolution of the project in the week 10 (second week of December) and (c) division and planning of the different stages of the project.

So it appears that we needed to finish quickly to fill the NaN values because our main objective was to have in one hand clean data and in the other experiment of `GFT` on simulated data. After that we could start testing our model with real values obtain through our cleaning to observe if we could obtain conclusive results on them.

4 Theoretical framework

In this section we will present the drift concept and the basic notions of graph theory as a descriptive basis of the sensor network, together with the mathematical tool called Fourier transform of graphs, used for the spatial-temporal description of the network by means of the spatial and space-time spectrograms.

4.1 Concept drift description

In a sensor network it is possible to witness problems in measurements, due to different factors such as: long use and wear of sensors, physical problems and large scales of devices [9], altering the network stability. The measurements usually change over time, even altering the distribution of data, making it difficult to create predictive models of the network because it is not possible to contrast it with the measured data. This problem is known as the concept drift [10].

Let $\mathbf{f} = [f_1(t), f_2(t), \cdots, f_n(t)]$ as a set of stochastic processes in each sensor, where each process is the composition of three signals: the original signal ($x_i(t)$), the noise ($n_i(t)$) and the signal due to the concept of drift ($d_i(t)$).

Figure 7: Different sample paths of a standard Brownian motion

Assuming that the sensor leads are independent, we can describe the $\mathbf{d}$ process as a standard Wiener (or standard Brownian motion) (Figure 7), with:

- $d_i(0) = 0$.
- $d_i(t) = d_i(t-1) + \delta_i(t), \quad \delta_i(t) \sim \mathcal{N}(0, \sigma_i^2(t))$ [9].
- For $0 \leq s < t \leq T, \quad d_i(t) - d_i(s) \sim \mathcal{N}(0, \sum_\tau^t \sigma_{i,\tau}^2 - \sum_\tau^s \sigma_{i,\tau}^2)$ (Gaussian stationary increment).
- For $0 \leq s_1 < t_1 < s_2 < t_2, \quad d_i(t_1) - d_i(s_1)$ and $d_i(t_2) - d_i(s_2)$ are independent (independent increment).

As mentioned, the concept drift is present in the signals that happen in a sensor network, being possible its analysis from the graph theory.

4.2 Graph Theory

Networks or graphs are finite collections of nodes or vertices (v_i) connected to each other by edges (e_{ij}). Usually, each line has an associated weight (w_{ij}) [11], which represents the relationship between the two nodes it connects, either its Euclidean distance, the pressure of a liquid or gas or the electric current. Additionally, can also be directed or undirected. The types of networks can be seen in the Figure 8.

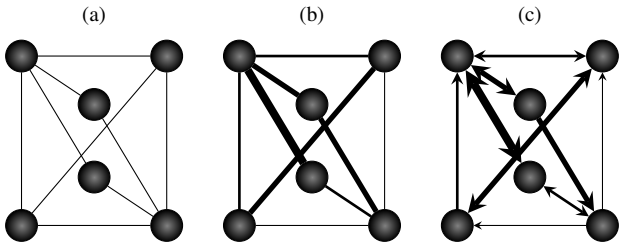


Figure 8: Graph diagrams with different configurations: (a) undirected binary, (b) directed weights and (c) directed weights.

One of the first problems solved by graph theory dates back to 1736, when Leonhard Euler solved the Königsberg Bridge Problem [12], and since then several problems have been solved with this mathematical tool. Some of the disciplines that use graph theory are: biological neural networks, transportation systems, computer application, electrical power infrastructures, social networks and sensor networks [13, 14, 15, 16, 17]. For more information on graph theory, see [13], section "Graph Theory: Analysis of the Brain as a Large, Complex Network".

As mentioned, a $\mathcal{G}$ network can be defined as $\mathcal{G} := (\mathcal{V}, \mathcal{E}, \mathbf{W}_\mathcal{G})$, where $\mathcal{V}, \mathcal{E}$ are the set of vertices and edges, respectively, $N = |\mathcal{V}|$ is the total of vertex and $\mathbf{W}_\mathcal{G}$ is the weight matrix of $\mathcal{G}$ graph. Depending on the type of configuration, there are 5 possible cases in the $\mathbf{W}_\mathcal{G}$ matrix:

- $w_{ij} \neq 0$ and $w_{ji} = 0$ if e_{ij} is one-directed on the way from i to j ($e_{ij} = 1$ and $e_{ji} = 0$),
- $w_{ij} = 0$ and $w_{ji} \neq 0$ if e_{ji} is one-directed on the way from j to i ($e_{ij} = 0$ and $e_{ji} = 1$),
- $w_{ij} = w_{ij}$ if e_{ij} is bi-directed with the same weight ratio,
- $w_{ij} \neq w_{ij}$ if e_{ij} is bi-directed without the same weight ratio,
- $w_{ij} = w_{ij} = 0$ if there is no connection between nodes i and j.

When $\mathbf{W}_\mathcal{G}$ is not naturally defined, we can use the definition explained in [18] (Equation 1)

$$\mathbf{W}_\mathcal{G} = \begin{cases} \exp\left(-\dfrac{[\text{dist}(i,j)]^2}{2\theta^2}\right) & \text{if } \text{dist}(i,j) \leq k \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

for θ, k parameters and $\text{dist}(i,j)$ a physical distance (e.g. Euclidean). On the other hand, we can describe the total edges by node in a diagonal matrix, called $\mathbf{D}_\mathcal{G}$. Furthermore, a representation of the Laplacian operator can be incorporated as the difference between $\mathbf{W}_\mathcal{G}$ and $\mathbf{D}_\mathcal{G}$, i.e. $\mathbf{L}_\mathcal{G} := \mathbf{D}_\mathcal{G} - \mathbf{W}_\mathcal{G}$ [19]. It is then possible to calculate the eigenvalues and eigenvectors to $\mathbf{L}_\mathcal{G}$ in order to obtain a new representation of $\mathcal{G}$, with properties similar to the frequency domain in the Fourier transform [6].

4.3 Graph Fourier Transform

Let's take $\mathbf{U}_\mathcal{G} = [\mathbf{u}_0, \mathbf{u}_1, \cdots, \mathbf{u}_{N-1}]$ as the eigenvectors matrix, and $\sigma(\mathbf{L}_\mathcal{G}) = \{\lambda_\ell\}_{\ell=0,1,\cdots,N-1}$ the entire spectrum of $\mathbf{L}_\mathcal{G}$ (theirs eigenvalues). We defined the GFT of a signal $\mathbf{f} = [f_0, f_1, \cdots, f_{N-1}] \in \mathbb{R}^{N \times T}$ on $\mathcal{G}$ ($\mathbf{f}$ is a matrix that represent the discrete signals in all the vertices with T instants of time) by the analogy with the Fourier Transform [17, 18, 6], defined in Equation 2.

$$\begin{gathered} \hat{f}_\ell := \langle f_\ell, \mathbf{u}_\ell \rangle, \qquad \ell = 0, 1, \cdots, N-1 \\ \text{GFT}\{\mathbf{f}\} = \hat{\mathbf{f}} := \mathbf{U}_\mathcal{G}^{*T} \mathbf{f} \end{gathered} \tag{2}$$

and, from the fact that $\mathbf{U}_{\mathcal{G}}$ is orthonormal, the Inverse Graph Fourier Transform ($\mathtt{GFT}^{-1}$) is equal to $\mathrm{GFT}^{-1}\{\hat{\mathbf{f}}\} = \mathbf{f} := \mathbf{U}_{\mathcal{G}}\hat{\mathbf{f}}$ [17]. Analogous to working with the Fourier transform, we can examine the signal in the graph by intervals, applying a filter with N translations (Figure 9).

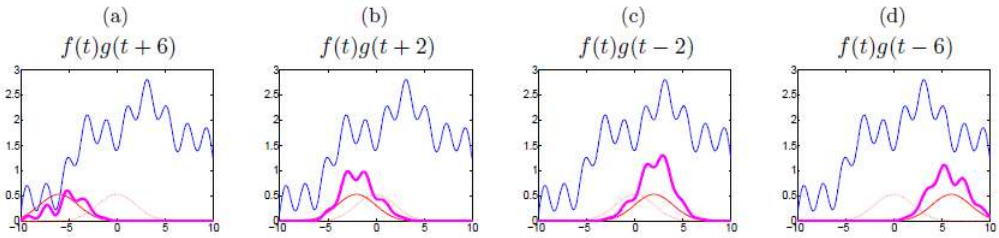


Figure 9: Representation of signal f (shown in blue) from the multiplication of a sliding window signal (shown in red) with itself, resulting in different window signals (shown in magenta), in order to calculate the Fourier window transform. Image taken from [6].

If we apply Equation 2 to each window signal, and organize the results in a matrix, a tool that analyses the energy density function in the frequency-temporal plane can be obtained. In the Figure 10b, an example of the **spectrogram** is presented, applied to a signal on graph Figure 10a.

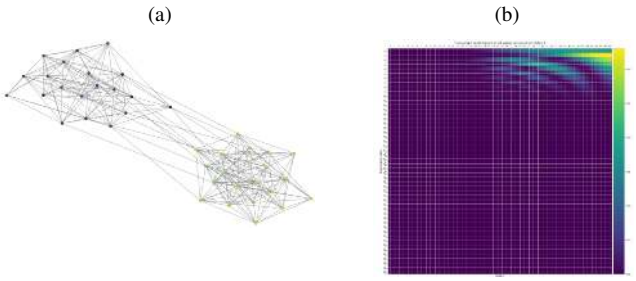


Figure 10: (a) Example of a graph with a random signal. (b) Spectrogram of the signal for vertex 0 of (a).

In terms of performing a deeper analysis on the behaviour of the signal in terms of space and time, we can initially establish a separate analysis of space and time, defining in the last dimension a $\mathbf{U}_{\mathcal{T}}$ described in Equation 3 [17].

$$\mathbf{U}^{*}_{\mathcal{T}}(t,k) = \frac{\exp(-j\omega_k t)}{\sqrt{T}} \qquad \text{with} \qquad \omega_k = \frac{2\pi(k-1)}{T} \tag{3}$$

and $t, k = 1, 2, \cdots .T$. Using Equation 3, The $\mathtt{JFT}$ is presented at the Equation 4

$$\mathrm{JFT}\{\mathbf{f}\} := \mathbf{U}^{*}_{\mathcal{G}}\mathbf{f}\bar{\mathbf{U}}_{\mathcal{T}} \tag{4}$$

In the same way, a space-time spectogram ($|Sf(i,j,k,l)|^2$) is defined as a 4D tensor where different windows centered on k-vertex and l-instant are manufactured, and the Equation 4 is applied for each (k,l) combination.

Spectrograms are a representation of the signals on a graph in the frequency domain and can be used in many fields of application. Therefore, it is interesting to combine this tool with outlier detection algorithms.

4.4 Outliers detection

A set of outliers in a sample are those that vary enough to suggest that they were generated by a different distribution [20]. There are multiple outlier detection algorithms, such as statistical-based [21], clustering [22], density [23] and distant [24] algorithms.

Let's $\mathbf{X} = \{X_1, X_2, X_3, \cdots, X_L\}$ the set of samples in space $\mathbb{R}^N$ (vectors), $\mathbb{R}^{N\times M}$ (matrices) or $\mathbb{R}^{N\times M\times\cdots}$ (tensors). Let $\mathbf{X_O}$ be the subset of $\mathbf{X}$ with the samples that represent the outlier cases. If the samples of $\mathbf{X_O}$ are sufficiently distant from $\mathbf{X}$, an outlier detection algorithm can be implemented. In this type of algorithm, the metric to evaluate the distance between two samples, such as the Manhattan (L1), Euclidean (L2) and Hamming distances, must be previously defined.

A common distance algorithm is usually the construction of a matrix representing the distance between all possible combinations of pairs of samples (X_i, X_j), with $i, j = 0, 1, \cdots, L$. The result of this algorithm is known as the **distance matrix** (Figure 11).

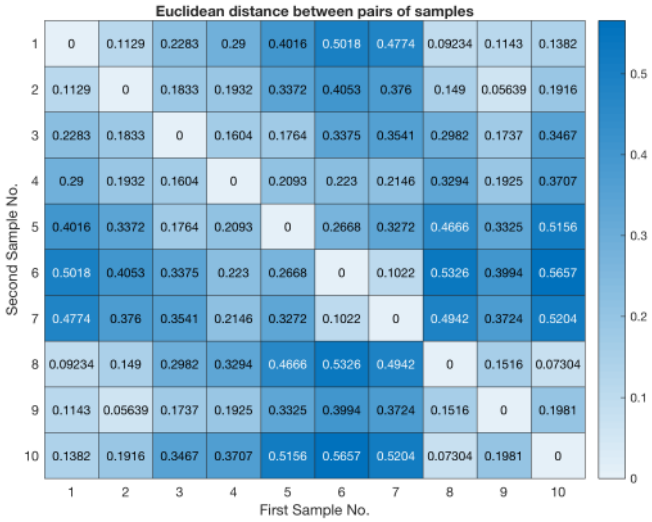


Figure 11: Example of a distance matrix. Image taken from Using the new Function heatmap to Display a Distance Matrix in Cluster Analysis webside.

General aspects of distance matrices:

- The matrix is symmetric ($d_{ij} = d_{ji}$).
- The diagonal is equal to 0 ($d_{ii} = 0$).
- If $d_{ij} > d_{jk} \approx 0$ and $d_{ik} > d_{jk}$, then node i has a greater distance to nodes j and k, which would symbolize that i is an outlier to j and k.

5 Data Understanding

Data quality is crucial, it assesses whether information can serve its purpose in a particular context. Then to respond to our problematic, it's necessary to work in our data in order to have good-quality data, indeed poor-quality data is often pegged as the source of operational snafus, inaccurate analytic and ill-conceived business strategies. Therefor a well understanding of the data is fundamental what are the different features, to what they correspond, are they accurate? Then we want to establish what feature at what time period could be the more interesting to analyse through our `GFT` to observe if we manage to identify beforehand a drifting sensor. So through data preparation we try to obtain as good-quality data as possible. In the data, overall, all the sensors seem to have the same behaviour, then the aim is to identity a sensor's behaviour that differs from others.

We had several sets of data, these are air quality data taken by sensors located in Pune (India), between November 2018 and October 2019. There were 52 sensors at the beginning but only 43 were still operational at the end. These sensors measure the concentration of various gases in the air of Pune (CO, CO_2, SO_2, NO, NO_2, O_3) as well as other air quality indices (AQI, PM2.5, PM10). The sensors measure this parameters several times per hour, although the time between each measurement is not fixed (on average one measurement is taken every 20 to 30 minutes). Moreover we do not have the real values of these different parameters, indeed the sensors operate with a system of minimum and maximum, so we have an estimate value most of parameters that can be more or less accurate: the difference between the minimum and maximum values of each parameter can vary a lot through time (Figure 12).

	AIR_PRESSURE	AQI	AQI_POLLUTANT	CATEGORY	CO2_MAX	CO2_MIN	CO_MAX	CO_MIN	DEVICEID	HUMIDITY	LASTUPDATEDATETIME	LIGHT	NAME	NO2_MAX	NO2_MIN	NO_MAX	NO_MIN	OZONE_MAX	OZONE_MIN	PM10_MAX	PM10_MIN	PM2_MAX	PM2_MIN
0	0.932	124.16	CO	MODERATE	365	310	117	69	02f7e7ee-29d6-c1c1-4d98-8f74981d9683-sensoragent	56.825	2019-05-29T04:39:26.000+05:30	1.312	ABC Farm House Junction_4	68	64	None	None	5	0	18	14	16	13
1	0.932	124.02	CO	MODERATE	365	310	116	69	02f7e7ee-29d6-c1c1-4d98-8f74981d9683-sensoragent	56.554	2019-05-29T04:24:26.000+05:30	1.209	ABC Farm House Junction_4	68	64	None	None	5	0	18	14	16	13
2	0.932	123.93	CO	MODERATE	365	310	116	69	02f7e7ee-29d6-c1c1-4d98-8f74981d9683-sensoragent	56.554	2019-05-29T04:09:26.000+05:30	1.455	ABC Farm House Junction_4	68	64	None	None	5	0	18	14	16	13
3	0.932	123.23	CO	MODERATE	365	310	116	69	02f7e7ee-29d6-c1c1-4d98-8f74981d9683-sensoragent	56.554	2019-05-29T03:31:02.000+05:30	1.585	ABC Farm House Junction_4	68	64	None	None	5	0	18	14	16	13
4	0.931	122.88	CO	MODERATE	365	310	116	69	02f7e7ee-29d6-c1c1-4d98-8f74981d9683-sensoragent	56.814	2019-05-29T03:15:59.000+05:30	1.379	ABC Farm House Junction_4	68	64	None	None	5	0	18	14	16	13

Figure 12: Features measured by sensors

In a first step, we proceeded to an analysis of the different parameters in order to choose on which parameter we will focus our experiments. Thus the main factors are that the measured values are relatively precise (small difference between minimum and

maximum values) so that the results we will obtain are not too much influenced by the measurement errors made by the sensors, in addition the gas or index chosen must be judged by the experts impacting when it is present in the air. Therefore, after cleaning the data we have visualised the difference between the maximum and minimum values of each parameter.

We observe that there are several possible cases, depending on the period and the gas, we can have values of min and max very close, quite close or very far. For the third case, it appears when the sensor is unable to evaluate the minimum gas concentration, therefore at that moment we have min values around zero although the max values can vary greatly. For our choice of parameter we want to avoid this phenomenon to occurs frequently. In addition, the most dangerous gases or indices when they are present too strongly in the air are co, NO_2, SO_2 and PM2. Our choice went to the PM2 index which measures the proportion of fine particles which are particles with a diameter of less than 2.5 micrometers. This index has the advantage of bringing together several gases or particles resulting from human activities that are dangerous for humans, moreover these particles are characterised by a very wide dispersion and can easily travel around the world.

Then we studied the variation of PM2 for all the sensors present at the end of the period and for the different periods that we had. We observed no significant differences between min and max the majority of the time, only missing values sometimes which mostly last a short time and are therefore quite easily to fill.

We tried to determine which period would be the most interesting to use the our space-time spectrogram. Since we wish to be able to identify in advance when a sensor will drift, we choose a period of 2 months to have enough point where a sensor seem to drift, we judge that a sensor seem to drift according to two condition. The first condition is that that sensor values are really different from other sensor values near to it. The second condition is that this sensor values are not in adequacy with measure values previous years at the same period.

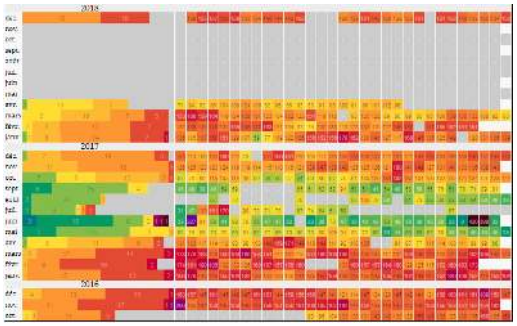


Figure 13: Historical data of PM2.5 since 2016 at Pune

According to Figure 13, we can see that the PM2 concentration varies greatly from month to month, however we notice that the highest values that have been measured are around 180 on many days, so these are not exceptional measurements. So if in our dataset we can identify a single sensor whose results are significantly above these values, we can assume that this sensor is drifting and use this period for our space-time spectrum analysis.

Finally, we find in our dataset a period that seems to correspond to the hypothesis we made that a sensor is drifting during the month of October 2019. Indeed, only one sensor displays values for this month that reach 300, in comparison we have a few other sensors, half a dozen whose values can go up to 200, and finally all other sensors display values below 100, limit before the air quality is considered worrisome for the inhabitants.

The different sensors had take measure in a not consistent way, so for the period we choose, for each hour we take the different measures taken and keep the average to obtain a new data where for each sensor we have a measure at each hour in order to make analysis. Then we verified the drifting character by comparing the values of the seemingly drifting sensor with its neighbourhood, we also confirmed by doing the same for sensors whose values reach 200 (Figure 14 and Figure 15).

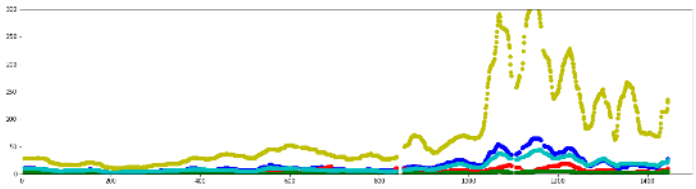


Figure 14: Comparison of seemingly drifting sensor in yellow with nearest sensors during September and October 2019

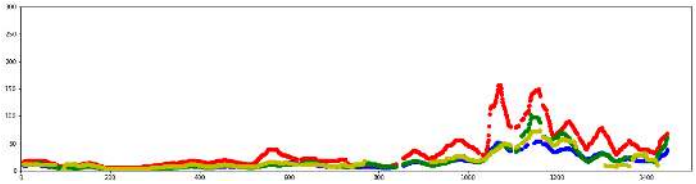


Figure 15: Comparison of sensor in red (that measured a relative high concentration of PM2.5) with neighbour sensors during September and October 2019

6 Data handling and methods

Air quality monitoring is applied to detect any concentration of substances that may have possible adverse effects on human health. However, this analysis is difficult because of the proportions of information that are often missing due to machine failure (derivative of sensors), routine maintenance, human error or external factors. Incomplete data sets can generate completely different results than those that would be obtained from a complete data set.

When dealing with incomplete data, there are 3 problems to be considered. First, there is a loss of information and, as a consequence, a loss of performance. Difficulties are encountered with respect to data processing, computation and analysis, due to data anomalies. Third, and most significantly, results are also biased by systematic variations between observed and unobserved data. One approach to solving incomplete data is the adoption of imputation techniques, for this you can use libraries such as fancyimpute that takes care of the use of imputation algorithms to handle the missing data. [25]

Due to these sensor failures, there is a large amount of NAN values in the data. The time interval sept - oct 2019 was defined, since in this interval it is observed that the sensors present drift, for which the temporal space analysis will be performed, however to perform a good analysis it is recommended that the data are not incomplete, so it is advisable to impute them. The annual hourly observation records of PM2.5 in Pune, India, are the data from 43 sensors that will be used throughout the project and the dataset to which imputation algorithms as linear interpolation and matrix factorization methods. The dataset consisted of PM2.5 material concentrations on an hourly time scale (hourly average) during the year 2019. A total of 25,284 gas values can be seen in Table 1, with a lack of information of 23.6% (6111 values).

Table 1: Characteristics of average PM2.5 data

Number of valid data points	19173
Number of missing data points	6111
Mean	11.254942
Standard deviation	4.423951
Percentiles :	
	25% - 8.0
	50% - 11.0
	75% - 14.0

6.1 Data Organization

We take a data set of m discrete time series. The time series could have missing elements. Then we express the spatio-temporal data set as a matrix $Y \in \mathbb{R}^{m \times f}$ with m rows (sensor locations) and f columns (discrete time intervals).

$$Y = \begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1f} \\ y_{21} & y_{22} & \cdots & y_{2f} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m1} & y_{m2} & \cdots & y_{mf} \end{bmatrix} \in \mathbb{R}^{m \times f}.$$

We then reorganize the data into a tensor. In the m dataset of time series we can observe missing values so. We split each time series in intervals of predefined length f. We express each partitioned time series as a matrix Y_i with n rows (hours) and f columns (discrete time intervals)

$$Y_i = \begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1f} \\ y_{21} & y_{22} & \cdots & y_{2f} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nf} \end{bmatrix} \in \mathbb{R}^{n \times f}, i = 1, 2, ..., m,$$

6.2 Linear Interpolation algorithm (LI)

Linear interpolation could be a useful methodology for linear polynomial exploitation curve fitting. It helps in constructing new data points in the data interval comprising the known values of a data set. Therefore, linear interpolation is a simple methodology for estimating a value from the vector of known data. It is useful for information prediction and data imputation.

Linear interpolation is useful while predicting a value between given data points. The strategy for linear interpolation is to use a line to join the observed data of given information on the positive and negative side with respect to the missing values. Often, linear interpolation is not successful for non-linear information. If the points within the information set vary by an excessive value, then linear interpolation may not provide an accurate estimate. In Equation 5 we can observe its equation [26].

$$\mathrm{y} = y1 + \frac{(x - x1)(y2 - y1)}{(x2 - x1)} \tag{5}$$

Equation 5 uses the coordinates of two given values to find the curve of best fit as a straight line. This will then give any unknown value of y at a known value of x [26].

6.3 Bayesian probabilistic matrix factorization (BPMF)

The idea behind this type of models is based on recommendation models, a user's preferences are determined by a minimal variety of unobserved factors. In a linear model, an associated user's rating is modeled by the inner product of a vector of item factors and a vector of user factors.It is described that N users assign to M articles or items is modeled by the product of a D × N matrix of user coefficients U and a D × M matrix of factors V. Learning in this model is performed by maximizing the log-posterior over the item and user features with fixed hyperparameters.The conditional distribution over the observed ratings are given by [27]:

$$p(U|\mu U, \wedge U) = \sum_{i=1}^{N} \mathcal{N}(U_i|\mu U, \wedge U^{-1}) \tag{6}$$

$$p(V|\mu V, \wedge V) = \sum_{i=1}^{M} \mathcal{N}(V_i|\mu V, \wedge V^{-1}) \tag{7}$$

Then, a Gaussian-Wishart priors is placed on the hyperparameters of the user and items. W is the Wishart distribution

$$p(\Theta_U|\Theta_0) = p(\mu U|\wedge_U)p(\wedge_U) = \mathcal{N}(\mu U|\mu_0, (\beta_0\wedge_U)^{-}1)W(\wedge_U|W_0, \nu_0)$$
$$p(\Theta_V|\Theta_0) = p(\mu V|\wedge_V)p(\wedge_V) = \mathcal{N}(\mu V|\mu_0, (\beta_0\wedge_V)^{-}1)W(\wedge_V|W_0, \nu_0)$$

6.4 Bayesian temporal matrix factorization (BTMF)

We can obtain a representation of a matrix $Y \in R^{N\times T}$ in a spatiotemporal setting, one can factorize it into a spatial factor matrix $W \in R^{R\times N}$ and a temporal factor matrix $X \in R^{R\times T}$ following general matrix factorization model [28]:

$$Y \approx W^T X \tag{8}$$

$$y_{i,t} \approx w_i^T > x_t, (i,t), \tag{9}$$

The vectors w_i and x_t refer to the i-th column of W and the t-th column of X, respectively. The standard matrix factorization model is a good model approach to handle the missing data problem; however, it cannot capture the time dependencies among the different columns of X, which are crucial in modeling time series data. To further determine the temporal dependencies, a VAR regularizer on X is introduced in TRMF [28].

$$x_{t+1} = \sum_{k=1}^{d} A_k x_{t+1-h_k} + \epsilon_t \tag{10}$$

6.5 Performance indicators

Two performance indicators were used to evaluate the imputation methods are, the Mean Absolute Percentage Error (`MAPE`) and the Root Mean Square Error (`RMSE`). Synthetic and observed data were compared to select the best method of estimating missing values [29].

$$\mathrm{MAPE} = \frac{1}{N}\sum_{i=1}^{N}|P_i - O_i| \tag{11}$$

The MAPE is the average difference between the predicted and actual data values, and is given by Equation 11, MAPE goes from 0 to infinity and a perfect fit is obtained when MAPE = 0 [29].

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{N}(P_i - O_i)^2}{N}} \tag{12}$$

The RMSE is one of the most commonly used measures of success for numerical prediction. Its value is computed by Equation 12. The smaller the RMSE value, the better the performance of the model [29].

6.6 Experimental results and analysis

The methods of Linear Interpolation (LI), Bayesian Probabilistic Matrix Factorization (BPMF), Bayesian Temporal Matrix Factorization (BTMF) and Matrix Comprexion (MC) were compared with each other, to measure the performance of imputation of the methods, the data presented in Sub-section 6.7 is used as a ground truth for testing the experiments. Data were randomly generated in which the Percentage of Missing Data (PMD) ranged from 5% to 30%. The higher the PMD, the more difficult the imputation will be. Then the data imputation methods are applied to the incomplete data in order to estimate the missing values from the observed values.

The data of the experiment is composed of 43 sensors from the city of Pune, India, to which a percentage of missing values will be applied , the imputed values and the ground truth values are compared with the area unit to judge the performance of the imputation. The higher the PMD, the more difficult the imputation will be. Then the data imputation methods are applied to the incomplete data in order to estimate the missing values from the observed values. Finally, the imputed values and therefore the ground truth values are compared with the purpose to evaluate the performance of the models and the imputed data. To reduce possible bias, the performance indicators explained in Sub-section 6.5 and methods are applied every 100 points in the time series and the average of these results is recorded. After comparing the performance of the models, the algorithm that presents a better performance to the original data of the average PM2.5 gas was applied.

In this study, PM2.5 gas will be considered. The complete data matrix consists of 43 rows and 588 columns, thus generating a matrix with a total of 25,284 points. Each row refers to a sensor that monitors air quality, and each column represents the readings from all monitoring stations at a particular time point.

6.7 Characteristics of missing data

Table 2 shows the descriptive statistics of all missing data samples 5%, 10%, 20%, 30% . The average varies little with the percentage of values removed, and is seen to be worse than the median.

Table 2: Characteristics of average PM2.5 data in different percentages of missing data

Percentage of missing data	**5%**	**10%**	**20%**	**30%**
# Valid data points	5087	5087	5087	5087
# Missing data points	240	490	1022	1519
Mean	8.688822	7.714258	7.512611	7.179792
Standard deviation	11.569303	10.038835	10.112278	9.973181
Min	0.0	0.0	0.0	0.0
Max	205.0	205.0	205.0	201.0
Percentiles :				
	25% - 3.0	25% - 3.0	25% - 3.0	25% - 2.0
	50% - 5.88	50% - 5.0	50% - 5.0	50% - 5.0
	75% - 10.92	75% - 9.0	75% - 9.0	75% - 8.0

6.8 Imputation error comparison

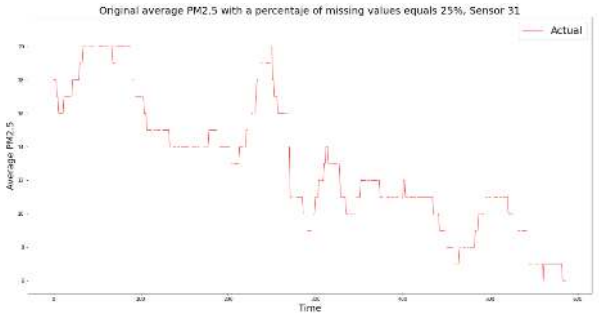
The imputation performance of various ways once PMD equals 5%, 10%, 20%, 30% is shown in Table 3 three wherever we show the mean values of RMSE and MAPE performance indicators across the tests.

In Table 3 we can observe that the method that has the best performance BPMF because it has the lowest error for all the percentages of missing data in the dataset, however BTMF shows excellent performance as well.

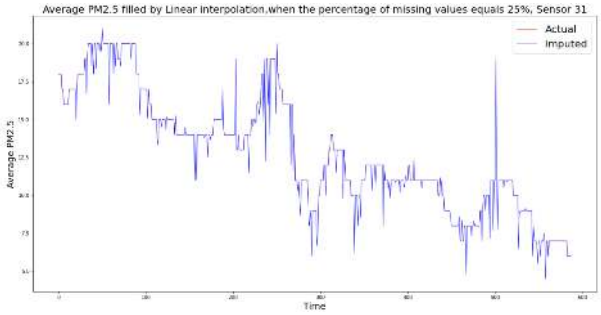
Table 3: Performance of methods with different percentages of missing values

P	Method	MAPE	RMSE
5%	BPMF	0.0489	0.52332
	BTMF	0.0407	0.6898
	LI	0.0612	0.2474
10%	BPMF	0.0540	0.6323
	BTMF	0.0647	0.9020
	LI	0.4335	3.4417
20%	BPMF	0.0603	0.9041
	BTMF	0.0837	0.9933
	LI	0.5134	4.7748
30%	BPMF	0.0554	0.6727
	BTMF	0.0635	0.8243
	LI	0.6431	2.1225

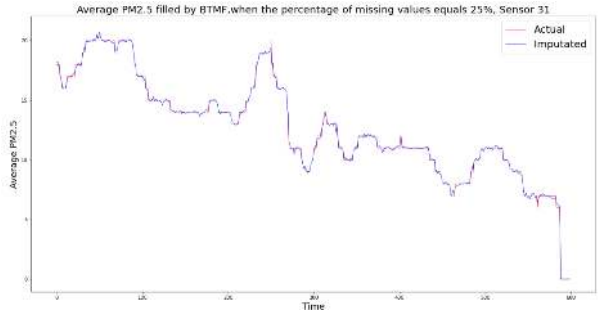
(a)



(b)



(c)



(d)

Figure 16: (a) Original data average PM2.5 with a percentage of 25% missing data (b) Filled data with linear interpolation method, (c) Filled data with BTMF, (d) Filled data with BPMF imputation method

In Figure 16a we can observe the time series of the original data with a PMD of 25% for sensor 31, on the other hand, in the images Figure 16b Figure 16c Figure 16d are the graphs of the time series with the predicted data for the sensor 31 with a percentage of missing data of 25%. The behaviour of the graphs with the predicted data is similar, however, the performance indicators variate as we can see in Table 3,we can see that the methods that are closest to the shape of the original graph are Figure 16c Figure 16d .

From the results, it can be inferred that, as the percentage of lost values continues to increase, so do the estimation errors. This occurs because the information available for imputation is reduced as the percentage of missing data increases. The variation of the imputation error with respect to the percentage of missing values is shown in Table 3 . It is also observed that for LI model the estimation error may vary more and the error indicators are usually greater than the other methodss, in BPMF and BTMF methods we observe that the error is quite stable and we still get good performance with high percentage of missing data.

After having made an adjustment to the network signals with the data imputations, it is possible to use them to observe their characteristics and their behavior in each sensor. Before this, we will exemplify the use of different tools in networks and synthetic signals, in order to understand in predictive models the behavior of these tools, to facilitate the understanding of the results when implementing these signals.

7 Graph simulation

In order to analyze and design a methodology for drift detection of a sensor in a network, a simulation of a network was made given a group of verticex $V = V_1, V_2, ...V_x$ with $x = 20$ and a path graph of $T = 15$ instants as shown in Figure 17. Following the example presented in [6], a test signal was created based on the eigenvectors of the spacial and temporal graph as a matrix multiplication between the signals generated in both graphs. In space, $\mathbf{u}_2^G$ and $\mathbf{u}_{11}^G$ eigenvectors was used to create the signals of group 0 and 1, respectively. In time, we used the $\mathbf{u}_3^T$ eigenvector for the first half of time and $\mathbf{u}_{12}^T$ for the second one.

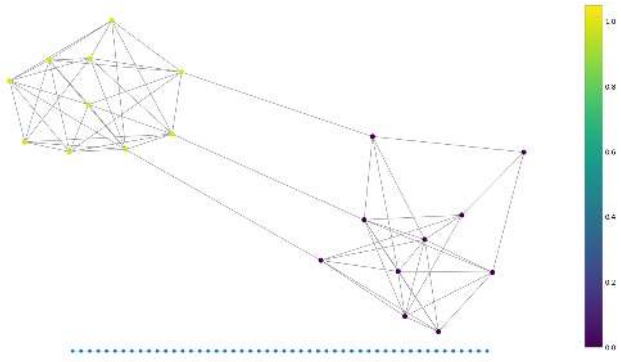


Figure 17: Synthetic graph

Figure 18 shows the signal in the spatial graph at instant $t = 4$ and its spectrogram. In Figure 18a we can see how half of the vertices have a higher frequency than the second half of vertices. In Figure 18b we observe the spacial spectrogram of the graph, where much of the energy is concentrated in the eigenvalues chosen above (λ_2^G and λ_{11}^G).

(a) (b)

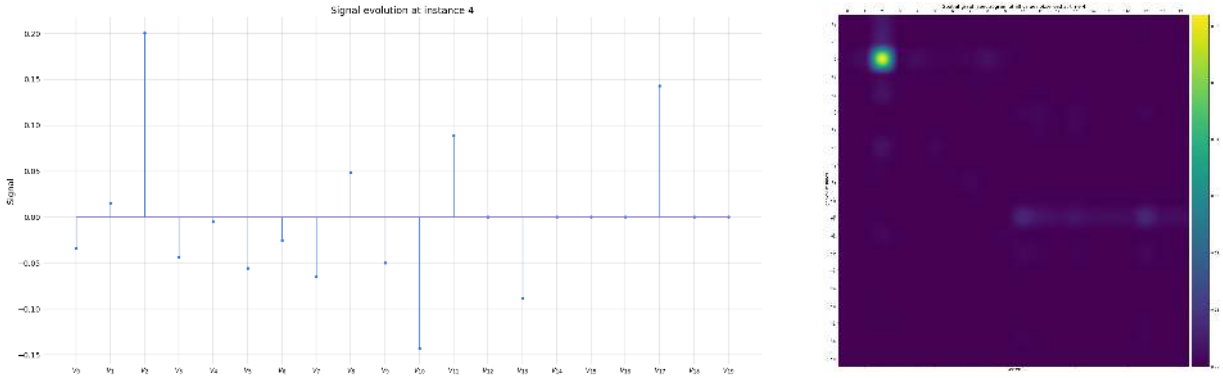


Figure 18: (a) Signal in spatial graph at instant $t = 4$. (b) Spectrogram in spacial graph at instant $t = 4$, with x-axis as the vertex number and y-axis as their eigenvalues.

Figure 19 shows the signal in the time graph at vertex 9 and its spectrogram. In Figure 19a it can be seen how half of the time there is a low frequency and then in changes to a bigger one. In Figure 19b it is observed the time spectrogram of the time graph, much of the energy is concentrated in the eigenvalues chosen above (λ_3^T and λ_{12}^T)

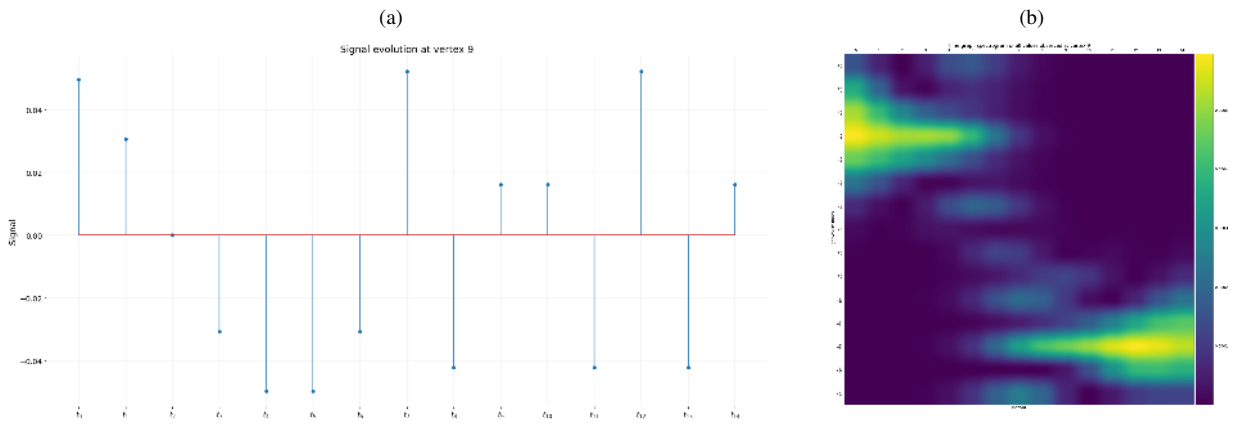


Figure 19: (a) Signal in time graph at vertex 9. (b) Spectrogram in graph time at vertex 9, with x-axis as the instants in time and y-axis as their eigenvalues.

Following the analysis, the information about the spectral behavior of the two graphs can be observed in joint spectrogram, which allows to analysis the behaviors of the network in space-time domain. Figure 20 show the spectrogram result for vertices 2 and instance 14 which belong to two different signals groups based on the eigenvectors $\mathbf{u}_2^G$ and $\mathbf{u}_{11}^G$ in space, and in time the signal changes in frequency according to eigenvectors $\mathbf{u}_3^T$ and $\mathbf{u}_{12}^T$.

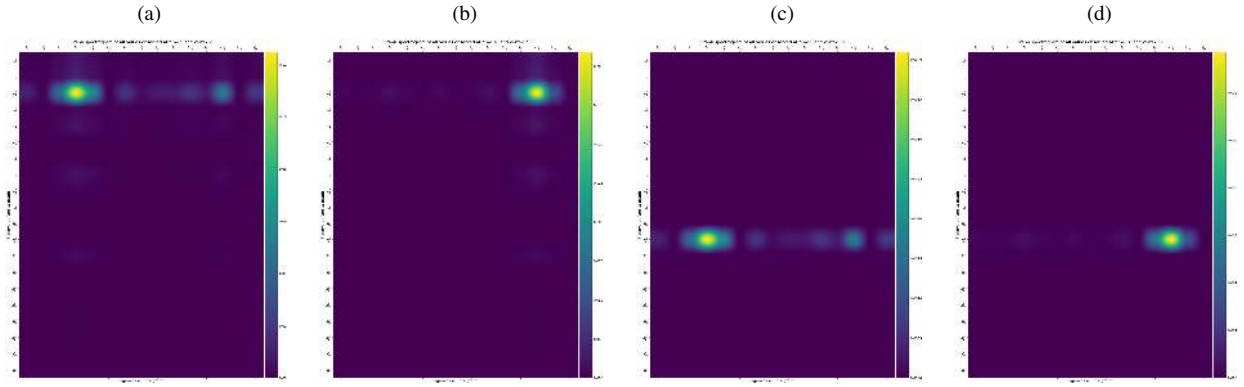


Figure 20: Joint spectrogram for $V = 2$ (first group) at (a) $t = 5$ and (b) $t = 10$. Joint spectrogram for $V = 14$ (second group) at instance (a) $t = 5$ and (b) $t = 10$. For all graphs, x-axis are the eigenvalues in the time graph and y-axis are the eigenvalues of space graph.

In order to analyze the behavior of the network when a drift happens in a sensor, we modified the signal in vertex 4 (belonging to the first group of the network) from instant 7, taking the signal as a Wiener process. The Figure 21a shows the generated signal and its respective spectrogram over time (Figure 21b), where is observed how the signal frequency decreases (almost constant) after the drift, and the highest energy is observed at the lowest eigenvalues.

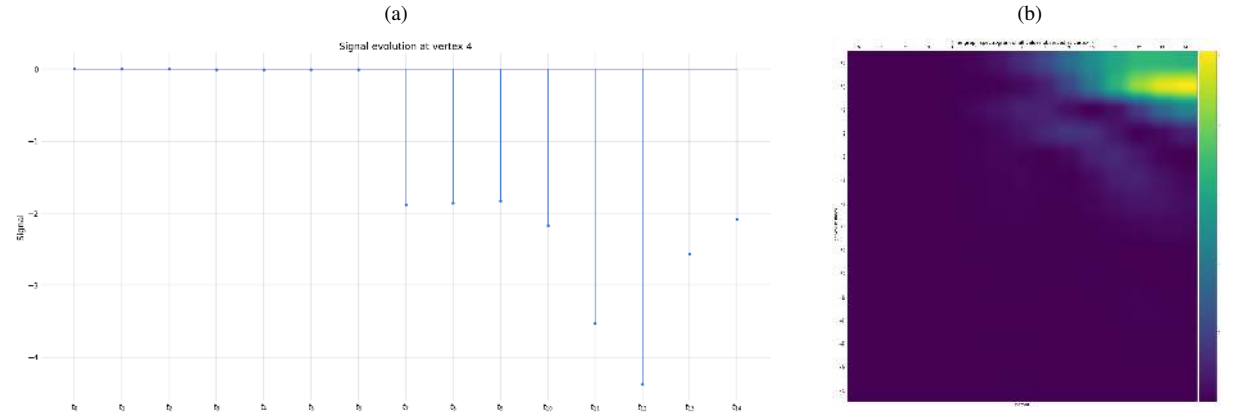


Figure 21: (a) Signal in time graph at vertex 4, drift occurs after instant $t = 7$ (b). Spectrogram in graph time at vertex 4, with x-axis as the instants in time and y-axis as their eigenvalues.

7.1 Drift detection application

In this section we make use of the algorithm designed in Sub-section 4.4 in order to find the damaged vertex and the instant from which the drift occurs. We take as sample of $\mathbf{X}$ set the *faces* of the joint spectrogram that lie within the interval $[v_i - s, v_i + s]$ (centered at the vertex v_i) in space and $[t_j - t, t_j + t]$ (centered at the instant t_j) in time. Explained another way, each sample $X_{(v_i,t_j)} \in \mathbb{R}^{V\times T\times(2s+1)\times(2t+1)}$ of the set $\mathbf{X}$ is a subset of the joint spectrogram, centered on the coordinate pair (v_i, t_j).

The Figure 22 shows the distance matrix, in which it is possible to observe that the maximum value corresponds to vertex 4 from instant 12 approximately.

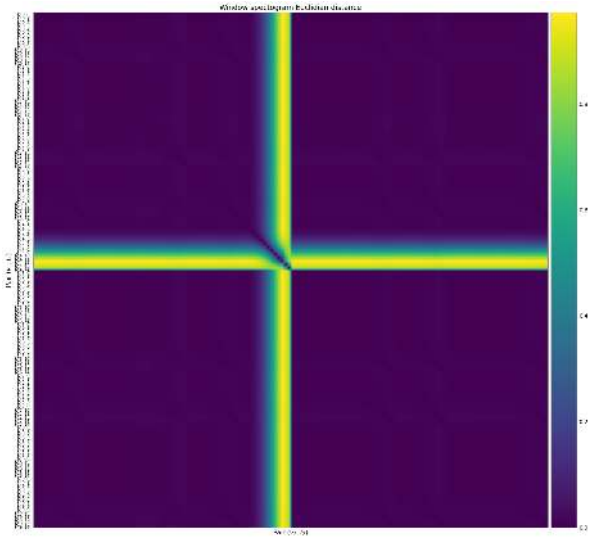


Figure 22: Distant matrix for the simulation example.

With this information it is possible to observe the vertex that presents the damage, however the instant from which the damage occurs is an approximation, so it is decided to apply the unsupervised maching learning algorithm Gaussian mixture (`GMM`) [30] in order to find the distribution of instants of all vertices. Figure 23 presents the result of the clusters, it is observed that in group 5, the samples of vertex 4 are found from instant 8, which coincides more with the original behavioral state of the signal in which the drift starts from instant $t = 7$.

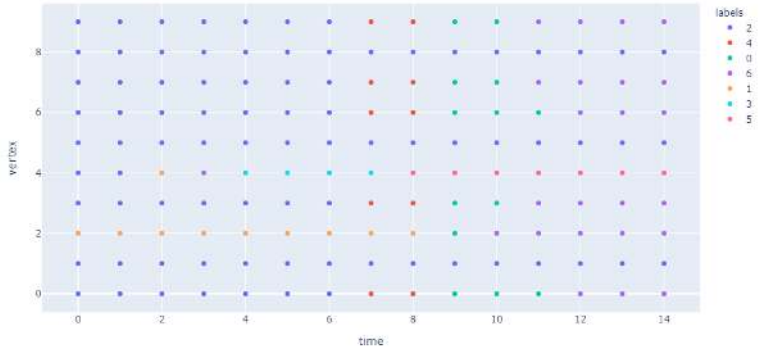


Figure 23: Result of applying `GMM` in distant matrix.

8 Graph construction

In order to analyse the behaviour of the air quality measurement sensor network in the city of Pune, information on the physical distribution of these sensors is available. There is an adjacent matrix with information about the distance between sectors (Figure 25). The Figure 24 shows the Probability Density Function (`PDF`) and Cumulative Distribution Function (`CDF`) of the distances in order to analyse and choose the limit to establish the weight between connections (Equation 1). It can be observed that the average value of the connections is between 5 and 7 and there are few connections greater than 17. If we take a threshold of $k = 5$ that is, connections greater than this value would be eliminated, 50% of the existing connections would be maintained, and we focus on the sensors that have a certain closeness in space.

(a) (b)

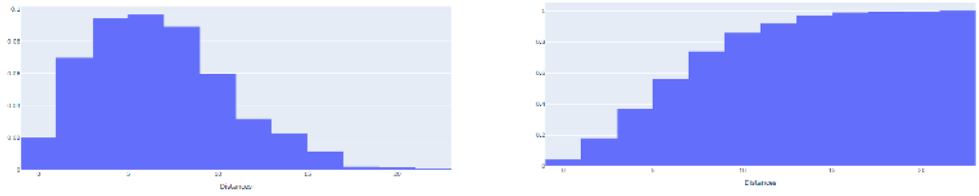


Figure 24: (a) Probability density function and (b) cumulative distribution function of distances.

(a) (b)

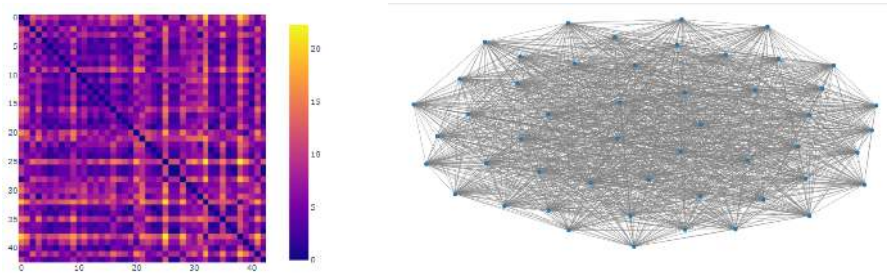


Figure 25: (a) Original adjacent matrix (b) Original Graph

Once the equation Equation 1 for $k = 6$ and $\theta = 4$ is applied, a more simplified version of the original network is obtained, in which more weight is given to the connection between close vertices. The Figure 26 presents the result obtained.

(a) (b)

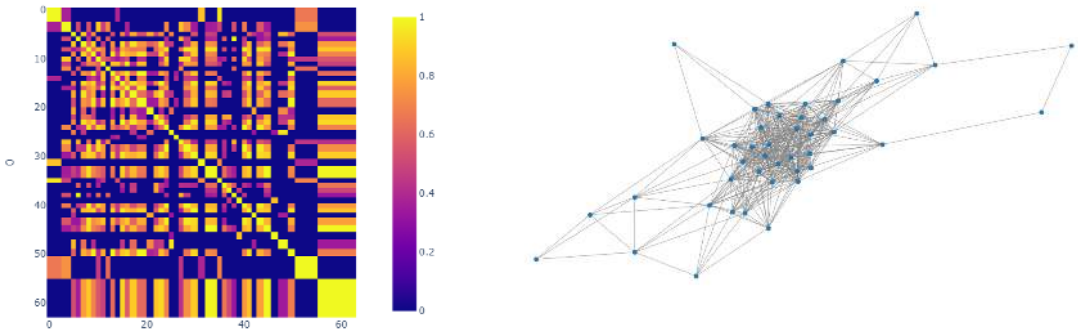


Figure 26: (a) Normalised adjacent matrix (b) Normalised Graph

In order to limit the interactions between the vertices of the network, an algorithm is applied to limit the connections of each vertex to ensure that there are only edges between vertices really close to each other, limiting the neighbors to 5 per vertex resulting in the network presented in the Figure 27.

(a) (b)

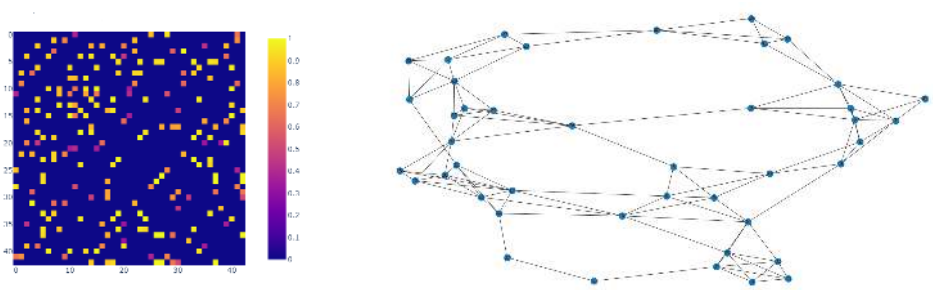


Figure 27: (a) Adjacent matrix neighbor algorithm (b) Graph neighbor algorithm

8.1 Graph organisation

Since sensors measure air quality in a particular zone, it is expected that nearby sensors will present similar measurements. If a sensor presents a very different measurement from one of its neighbors, this could be an indication of a drift in the sensor, that is why , it is important to achieve a network organisation in order to get a better view of the really "close" nodes and improve the performance of network algorithms. Several methods have been proposed in the literature for this purpose, such as bandwidth reduction [31] and spectral clustering [32]. In this section both algorithms will be explored and the best result will be chosen.

8.1.1 Spectral clustering

Clustering is one of the most popular techniques used for exploratory data analysis, with applications ranging from statistics, computer science, biology to social sciences or psychology. In general the principal aim is to find identify groups of "similar behaviour" in data [32]. Taking into account the graph theory presented in Sub-section 4.2, the approach in [32] [33] establish that under small perturbations and one can detect clusters by running k-means on the rows of the matrix of eigenvectors of the small eigenvalues of the graph Laplacian. The steps for the (unnormalized) spectral clustering are described in [32].

The above-mentioned algorithm is applied to the adjacent matrix shown at Figure 26. In order to determine the number k of clusters, several tests are performed with the first 10 eigenvectors, selecting a $k = 10$ from the inertia graph (Figure 28a) and thus making the adjacency matrix (Figure 28b) and the clustered graph (Figure 28c).

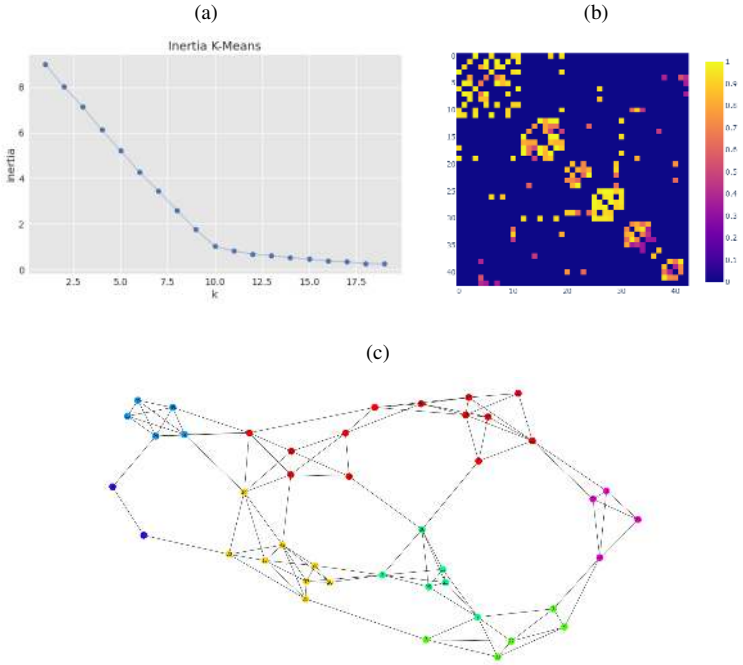


Figure 28: (a) Inertia K-means (b) Adjacency matrix after spectral cluster (b) Graph after spectral cluster

8.1.2 Bandwidth reduction

The bandwidth minimization problem can be viewed in the context of both graphs and matrices. From graph point of view the problem consists of finding a special labeling of vertices which minimizes the maximum absolute difference between the labels of adjacent vertices. From matrix point of view, the minimization problem consists of finding a rows/columns permutation that keeps most nonzero elements in a band that is as close as possible to the main diagonal of matrix . Currently there are multiple algorithms to achieve bandwidth reduction as shown in [34]. One of the most famous is the Cuthill Mckee (`CM`) algorithm.

The Cuthill Mckee algorthm [31] is used for reordering a space symmetric square matrix, the sparsified version of a matrix is a matrix in which most of the elements are zero. . Given a symmetric matrix A, a bandwidth-reduction ordering aims to find a permutation P so that the bandwidth of PAP^T is small. The `CM` algorithm consists in choosing a peripheral vertex (grade 1) and generate levels for all other vertices. Then, the vertices are selected depending on their degrees, i.e. ascending. The final result depend on the starting vertex, even if more vertices have same degree [35]. The Reverse Cuthill-Mckee Algorithm (`RCM`) [35] is the same as the `CM`, the only difference is that the final indices obtained using the `CM` are reversed in the `RCM`. which is applied in this work to reorganize the graph obtained in the Figure 26. Figure 29 is shown the result after aplying the `RCM`.

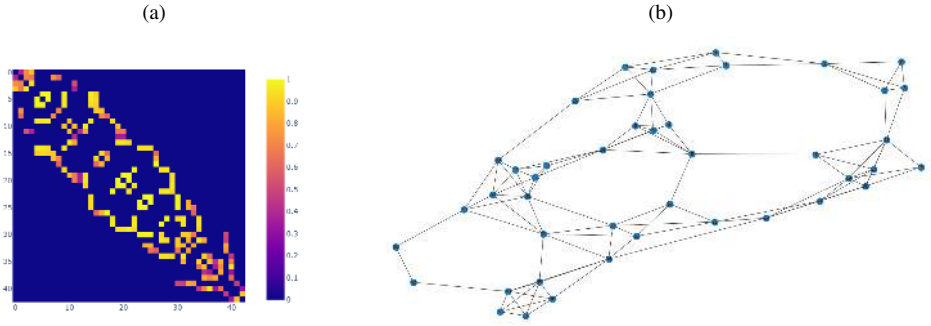


Figure 29: (a) Adjacent matrix after bandwidth reduction (b) Graph after bandwidth reduction

From the result obtained when applying both methods (Figure 29 and Figure 28) it is observed that with the algorithm `RCM` a smaller bandwidth is obtained, since most of the elements remain closer to the diagonal.

The Figure 29b presents the final network with which the analysis of the sensors' behaviour in the city of Pune will be performed. From this figure it is possible to extract a more realistic view about the real position of the sensors in the city, it is observed that most of them are located in nearby areas, therefore it could be easier to identify a failure in these sensors, by comparing the measures taken by this group in several time instants.

9 Application of graph theory and GFT to sensor networks

In this section, the graph theory is used to analyze the behavior of the sensor network of the city of Pune. Initially, several experiments are performed with simulated data that seeks to imitate a drift in a sensor, with this information we seek to highlight the usefulness of spectrograms to find changes in space and time. Then, using the real information about PM2.5 measurement in the sensors, treated in the Section 6, the experiment is carried out to identify the sensors that present failures (drift) and in which instant in time.

9.1 Experimental data analysis

To perform a spatial-temporal analysis of the measurements in the sensor network of the city of Pune, two graphs are available, one obtained in Section 8 which represents the proximity present between the sensors and a second graph (path) which represents the pollution measurements at different moments of time. There is a network of 43 sensors and 50 moments of time. In order to analyze the behavior in the sensor network in case of a failure in one of the sensors, a study of it is made using simulated signals for all the vertices, which a damage (drift) in vertex 4 at instant 20 is simulated as in Section 7. In space, $\mathbf{u}_5^G$ and $\mathbf{u}_{34}^G$ eigenvectors was used to create the signals of group 0 and 1, respectively. In time, we used the $\mathbf{u}_6^T$ eigenvector for the first third of time, $\mathbf{u}_{20}^T$ for the second one and $\mathbf{u}_{38}^T$ for the last part.

The Figure 30 shows the groupings made in the sensor network. Taking into account the physical proximity between sensors, three groups are made. And the same signal is generated for vertex in the same group.

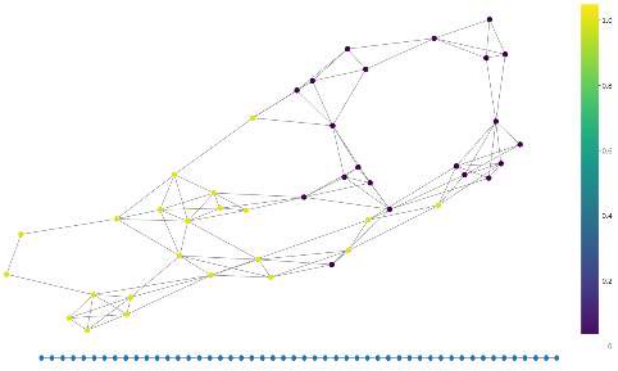


Figure 30: Grouping of vertex

The Figure 31 shows the signals generated for each vertex. The Drift is generated according to Section 4 and occurs in group one in the vertex 4 at instant 20

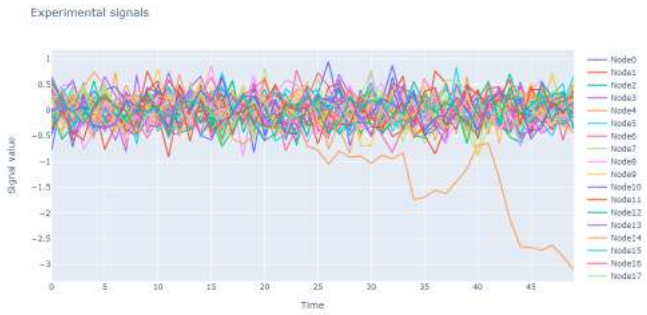


Figure 31: Signals in time generated for each vertex

As mentioned above, the spectrograms will be used to analyze the behavior of the network in different moments in time. Since there are two graphs in time (path) and space (connections between vertices) it is possible to perform the analysis of signals in both dimensions. The Figure 32 presents the spectrogram obtained in the space graph in time instant 10 and the signal in the graph at the given time. In the spectrogram (Figure 32b), high power values close to the eigenvalues chosen to generate the signal are observed.

(a) (b)

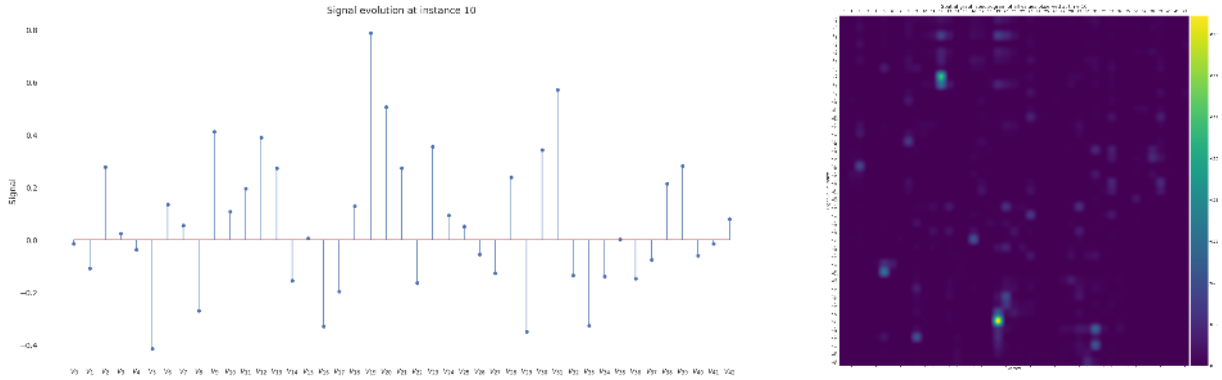


Figure 32: (a) Signal value at each vertex at time 10 (b) Space-spectrogram obtained in space graph at time 10, with x-axis as the vertex number and y-axis as their eigenvalues.

The Figure 33 and Figure 34 shows the resulted spectrograms in the time graph and the signal in time for nodes 6 and 4 respectively.

The vertex 6 is a neighbour (close in distance) of vertex 4, which in theory should have the same behaviour but because of the drift generated in instant 20 ,the signal is very different as is the spectrogram. Comparing the histograms Figure 33b and Figure 34b it can be seen that they differ greatly and it could be possible to estimate the time instant (x-axis) in which they stop resembling each other. It is observed that after drift, the spectrogram corresponding to vertex 4 presents greater magnitude in low frequency components (low eigenvalues), which is due to the fact that after drift the signal tends to increase constantly, contrary to the spectrogram of the neighboring vertex 6.

(a) (b)

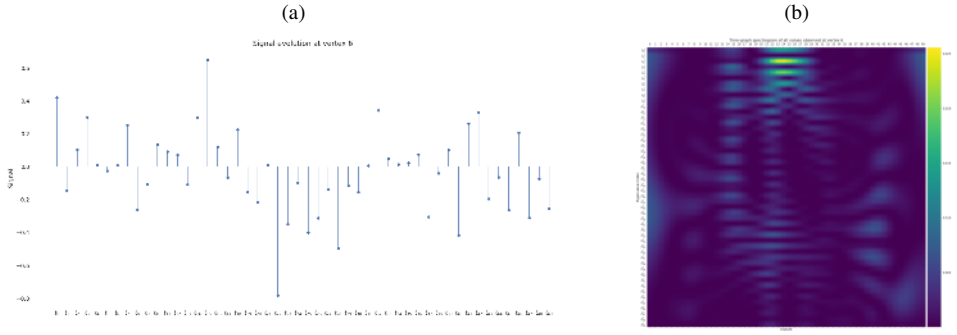


Figure 33: (a) Signal value at vertex 6 in time (b) Time-spectrogram obtained in time graph at vertex 6. The x-axes is the instants and the y-axis is their eigenvalues.

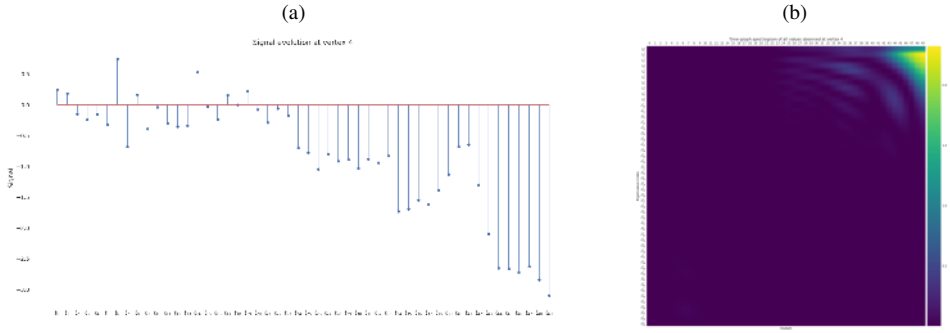


Figure 34: (a) Signal value at vertex 4 in time (b) Time-spectrogram obtained in time graph at vertex 4.The x-axes is the instants and the y-axis is their eigenvalues.

The summary information of all possible combinations of spectrograms in time and space is obtained in the four-dimensional spectrogram, which give a notion of variability in time and space jointly. In Figure 35 and Figure 36 are shown the space-time joint spectrogram observed for $k = [6, 4]$ and $l = [2, 30]$, in order to compare the result obtained in different moments of time (before the drift and after) for neighboring vertices. Now, the x-axis represents the time-eigenvalues and y-axis keeps the space-eigenvalues.

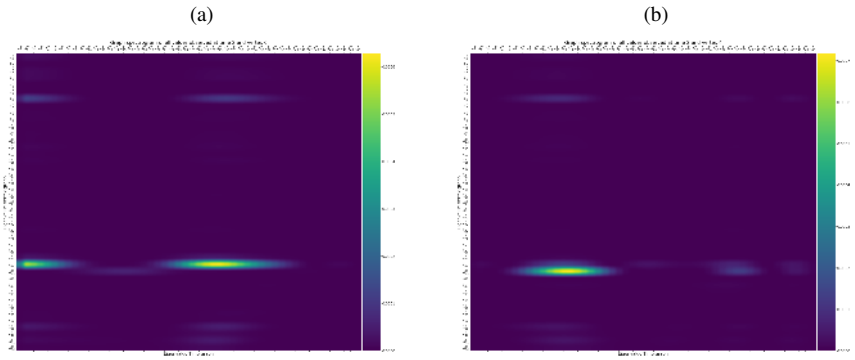


Figure 35: (a) Joint spectrogram (time - space) for vertex 6 at instant 2 (b) Joint spectrogram (time - space) for vertex 4 (damage vertex) at instant 2. (y-axes eigenvalues vertex graph, x-axes eigenvalues time graph)

Before the drift Figure 35, both vertices present a very similar behavior, both present high power components in space in the eigenvalues $\mathbf{u}_5^G$ and $\mathbf{u}_{30}^G$ and in time very similar values. As the drift advances, we see how a significant decrease in the high frequency time components occur, as a result of the change in the signal at one of the nodes Figure 36.

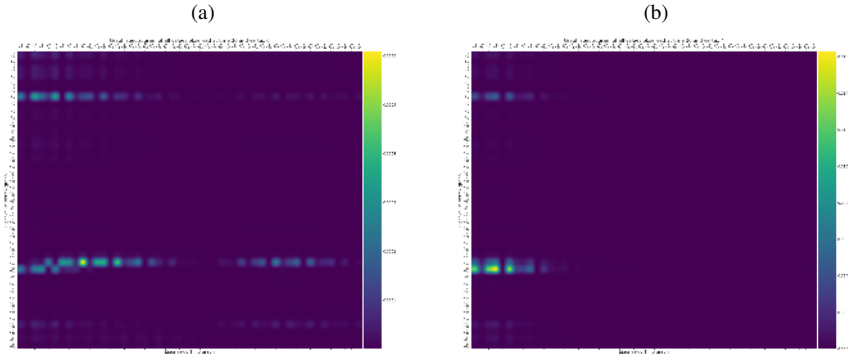


Figure 36: (a) Joint spectrogram (time - space) for vertex 6 at instant 30 (b) Joint spectrogram (time - space) for vertex 4 (damage vertex) at instant 30. (y-axes eigenvalues vertex graph, x-axes eigenvalues time graph)

9.2 Drift detection application

In this section we make use of the algorithm designed in Sub-section 4.4 as in Section 7.

The Figure 37 shows the distance matrix, in which it is possible to observe that the maximum value corresponds to vertex 4 from instant 30 approximately.

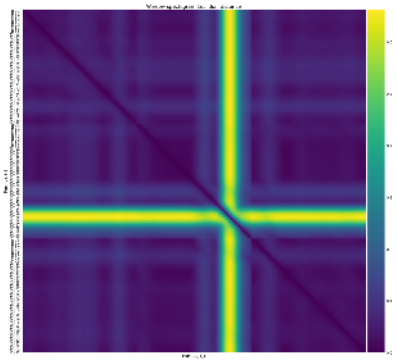


Figure 37: Distant matrix for the real graph with simulated signal

9.3 Real data analysis

According to the analysis made in the Section 5 there are some sensors that present measurements out of the ordinary of the PM 2.5 gas, periods between September and October 2019. After the missing data has been completed using the methods presented in the Section 6 , this final result will be used to test the use of spectrograms for drift detection, more precisely which sensor and at what time it happens.

To perform the analysis, a 40h window (moments in time) is chosen between August 22 and August 24, 2018 to analyse the behaviour of the sensors moments before and after the drift. The Figure 38 presents the signals from the sensors within the established period of time. It is observed that sensor 29 shows a drift after 18h on August 22.

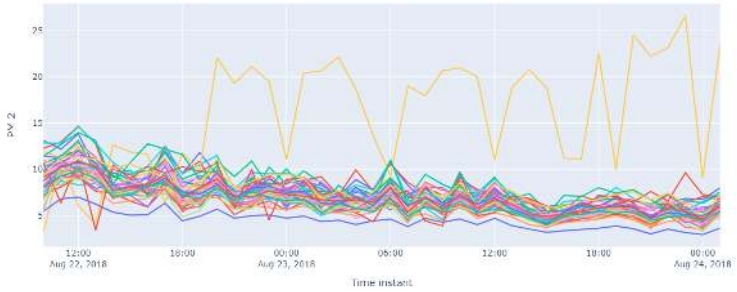


Figure 38: Signals from the sensors within August 22 and August 24. Drifting vector 29

Figure 39 present time-spectrograms of the sensors 22 and 29. It is possible to observe how the spectrograms of the network differ in time when an abnormal event occurs in the signal. However, since the signal does not present a high frequency in time, most of the energy is concentrated in the lower eigenvalues.

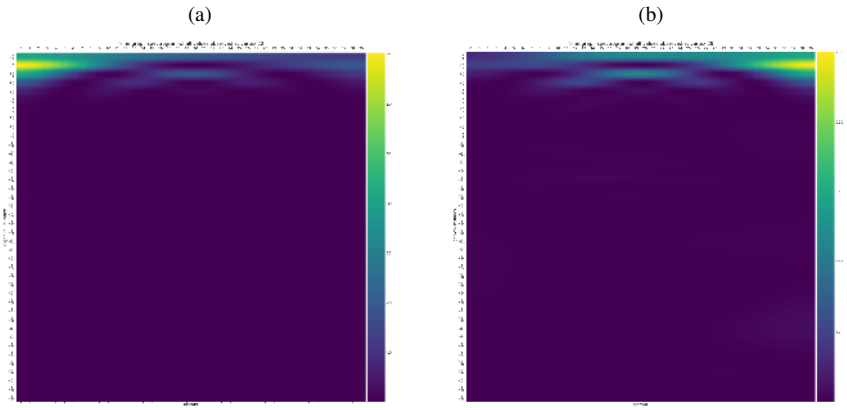


Figure 39: (a) Time-spectrogram obtained in time graph at vertex 22.The x-axes is the instants and the y-axis is their eigenvalues (b) Time-spectrogram obtained in time graph at vertex 29.The x-axes is the instants and the y-axis is their eigenvalues.

Figure 40 shows the joint spectrogram of nodes 22 and 29 at instant 37 (after drift).In Figure 40b it is observed that much of the energy is concentrated in higher eigenvalues in space, which could indicate the difference in the measurement of the vertex with respect to its neighbors.

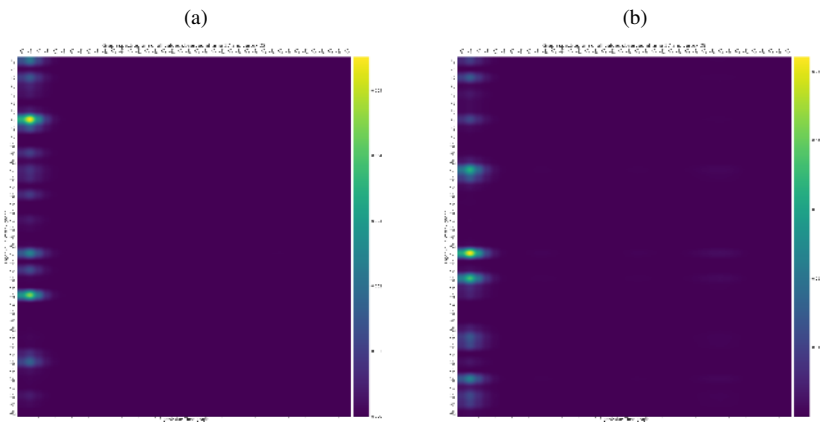


Figure 40: (a) Joint spectrogram (time - space) for vertex 22 at instant 37 (b) Joint spectrogram (time - space) for vertex 29 at instant 37. (y-axes eigenvalues vertex graph, x-axes eigenvalues time graph)

9.4 Drift detection application real data

In this section we make use of the algorithm designed in Sub-section 4.4 as in Section 7.

The Figure 41 shows the distance matrix, in which it is possible to observe that the maximum value corresponds to vertex 29 from instant 15 approximately.

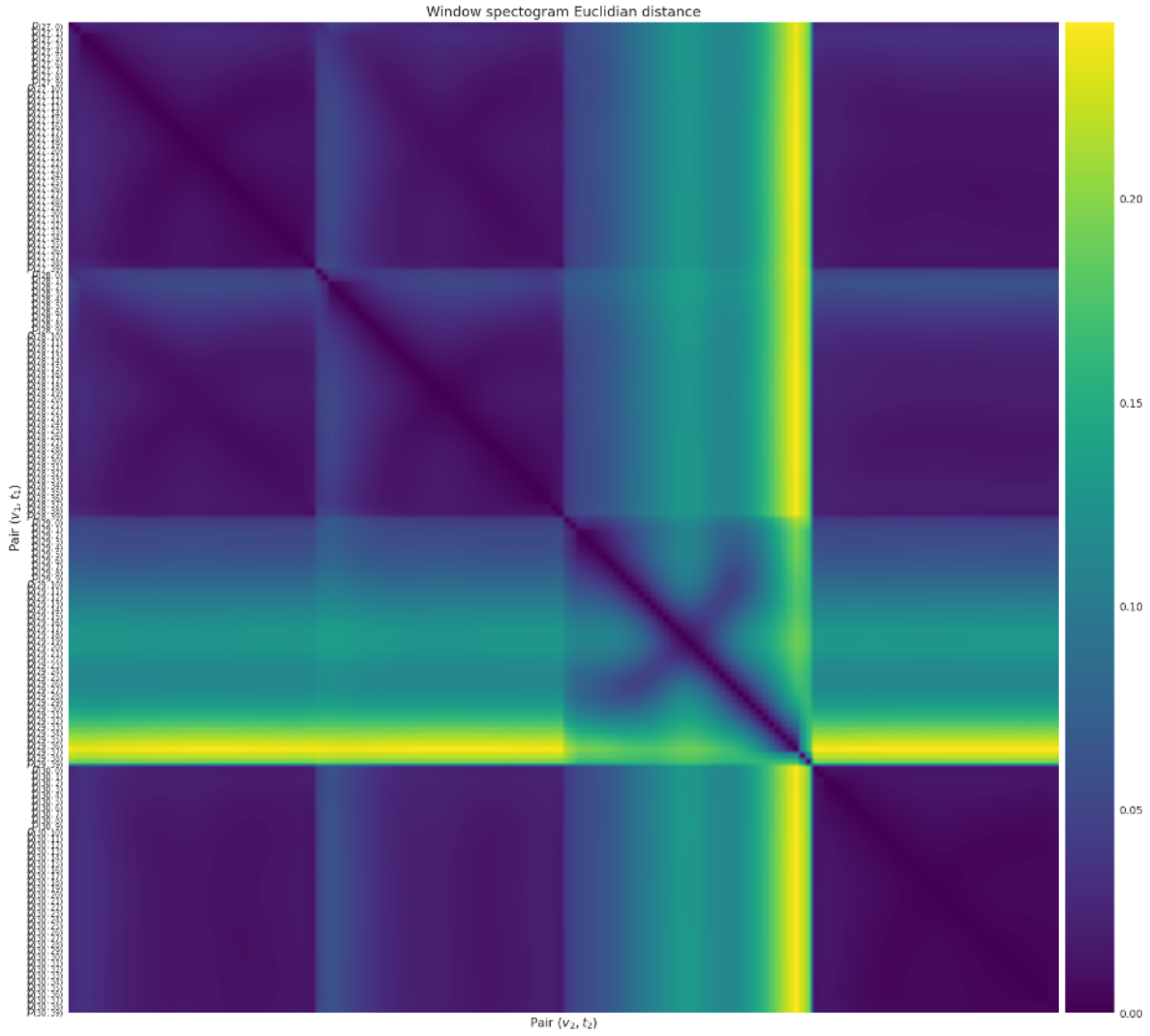


Figure 41: Distant matrix for the real graph with real signals

As in Section 7 the unsupervised maching learning algorithm `GMM` [30] is applied in order to find the distribution of instants of all vertices. Figure 42 presents the result of the clusters, it is observed that in group 2, the samples of vertex 29 are found from

instant 14, which coincides more with instant shown in Figure 38

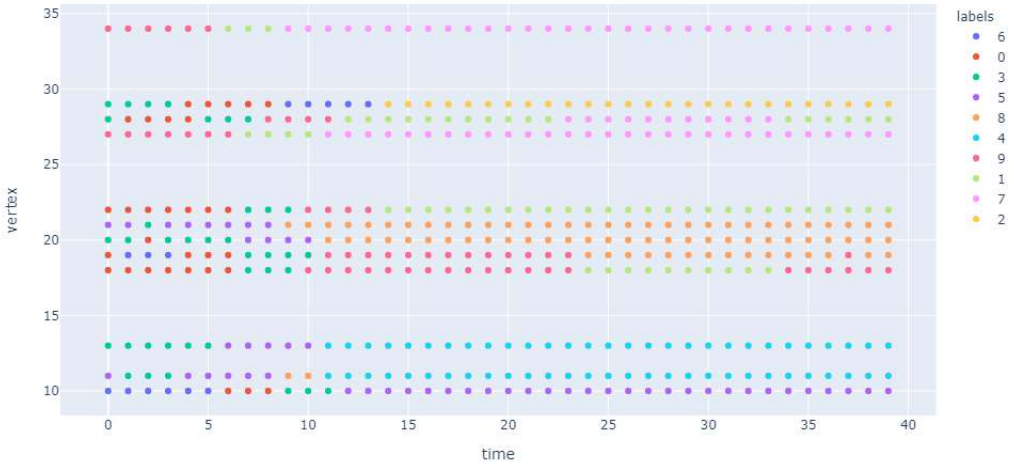


Figure 42: Result of applying `GMM` in distant matrix Figure 41

10 Conclusions

Thanks to the graph theory it is possible to extract an understanding of the phenomena that is happening in different moments of time. With the theory, multiple signals in a sensor network can be interpreted as a matrix, which will act as input to a matrix-based transformation system that represent the sensor network. That is, the signals are brought to another understanding of space whose basis lies in the eigenvectors of the matrix that characterizes the graph. What is interesting about this procedure is the interpretation of the spatial/temporal network with the union of the signals in a set, which allows the visualization of this information in a matrix. At the same time, it is possible to use tools that allow the analysis to be segmented in order to increase the specific understanding of the problem, in small windows where the study is important.

As evidenced in the work, one tool used for detailed analysis is the spectrogram, which provides the relationship between the frequency and time components of a signal. With the spectrogram, patterns can be observed and characteristics can be separated, for the later use of pattern detection algorithms, such as vertices with greater importance or atypical measurements (concept of drift). At the same time, it is important to understand that for a better understanding of the results obtained by the spectrogram, the network sort should be clarified to improve the neighboring sensors analysis as an ordered whole. To a greater extent, the organization can be executed with grouping algorithms as clustering techniques or from low distances with the use of adjacency matrices.

From the analysis made with simulated signals and the real one, it is observed the great influence of the frequency of the graph signal at the moment of finding abnormal behaviour using spectrograms, if the variation of the signals in time is low and therefore in space (measurement of sensors in a time t) the magnitude of the eigenvalues is concentrated in the small values, even after the drift.

The imputation of missing values is a very important part of the air quality information analysis. One way to build the use of the implicit structure that supports the information is closely associated with the performance of the imputation model. This exploration examines the issue of missing information retrieval, using a quality air pollution problem as a scenario. The issue has been developed and two commonly used information retrieval approaches, the Interpolation approach and thus the Matrix factorization approach. Simulations are conducted to test the performance of the intended `BPMF`, `BTMF LI`. The results of the simulations suggest that the `BPMF` and `BTMF` work properly in most of the cases.

It was determined that in the different algorithms the hyperparameters of the model have a great influence on the performance of the data imputation, so it is important to be very careful in these to obtain a good result, it is observed that the methods using as a basis the concept of matrix factorization have a good performance in the filling of data in a real database and with a large number of data , the processing time it takes to run these algorithms is not excessive.

Finally, we show that it is possible to detect the concept drift from the distance matrix of the joint spectrograms, which act as a feature decomposition tool and input to the distance matrix algorithm. In this new representation the values with higher magnitude represent the outliers of the samples set, being able to classify in two groups the samples with a similar distribution and those without (outliers).

In the future it is expected to use these tools to build a labeled database, and to apply some kind of supervised learning mechanism that allows (possibly in real time) to detect the sensors that are failing in a network from the signals in their spectral representation.

References

[1] A. Ortega, P. Frossard, J. Kovačević, J. M. F. Moura, and P. Vandergheynst, "Graph signal processing: Overview, challenges, and applications," *Proceedings of the IEEE*, vol. 106, no. 5, pp. 808–828, 2018.

[2] A. Sandryhaila and J. M. F. Moura, "Discrete signal processing on graphs: Graph fourier transform," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013, pp. 6167–6170.

[3] M. A. Erturk and L. Vollero, "Gsp for virtual sensors in ehealth applications," in *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2020, pp. 1683–1688.

[4] R. K. Jain, J. M. F. Moura, and C. E. Kontokosta, "Big data + big cities: Graph signals of urban air pollution [exploratory sp]," *IEEE Signal Processing Magazine*, vol. 31, no. 5, pp. 130–136, 2014.

[5] A. Loukas and D. Foucard, "Frequency analysis of time-varying graph signals," in *2016 IEEE Global Conference on Signal and Information Processing, GlobalSIP 2016 - Proceedings*, 2017.

[6] D. I. Shuman, B. Ricaud, and P. Vandergheynst, "Vertex-frequency analysis on graphs," *Applied and Computational Harmonic Analysis*, 2016.

[7] D. Kumar, S. Rajasegarar, and M. Palaniswami, "Automatic sensor drift detection and correction using spatial kriging and kalman filtering," in *2013 IEEE International Conference on Distributed Computing in Sensor Systems*, 2013, pp. 183–190.

[8] D. hui, L. Jun-hua, and S. Zhong-ru, "Drift reduction of gas sensor by wavelet and principal component analysis," *Sensors and Actuators B: Chemical*, vol. 96, no. 1, pp. 354 – 363, 2003. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0925400503005690

[9] Y. Wang, A. Yang, Z. Li, X. Chen, P. Wang, and H. Yang, "Blind drift calibration of sensor networks using sparse Bayesian learning," *IEEE Sensors Journal*, 2016.

[10] A. Tsymbal, "The problem of concept drift: definitions and related work," *Computer Science Department, Trinity College Dublin*, 2004.

[11] O. Sporns, "Graph theory methods: applications in brain networks," *Dialogues in clinical neuroscience*, vol. 20, no. 2, pp. 111–121, Jun 2018, 30250388[pmid]. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/30250388

[12] L. Euler, "Solutio problematis ad geometriam situs pertinentis," *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, vol. 8, pp. 128–140, 1736.

[13] F. V. Farahani, W. Karwowski, and N. R. Lighthall, "Application of graph theory for identifying connectivity patterns in human brain networks: A systematic review," 2019. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fnins.2019.00585/full

[14] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world9 networks," *Nature*, 1998.

[15] S. Boccaletti, V. Latora, Y. Moreno, M. Chavez, and D. U. Hwang, "Complex networks: Structure and dynamics," 2006.

[16] F. Schweitzer, G. Fagiolo, D. Sornette, F. Vega-Redondo, A. Vespignani, and D. R. White, "Economic networks: The new challenges," 2009.

[17] F. Grassi, A. Loukas, N. Perraudin, and B. Ricaud, "A Time-Vertex Signal Processing Framework: Scalable Processing and Meaningful Representations for Time-Series on Graphs," *IEEE Transactions on Signal Processing*, 2018.

[18] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," *IEEE Signal Processing Magazine*, 2013.

[19] A. J. Smola and R. Kondor, "Kernels and regularization on graphs," in *Lecture Notes in Artificial Intelligence (Subseries of Lecture Notes in Computer Science)*, 2003.

[20] F. Cao, X. Wu, L. Yu, and J. Liang, "An outlier detection algorithm for categorical matrix-object data," *Applied Soft Computing*, vol. 104, p. 107182, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494621001058

[21] C. E. Shannon, "The Mathematical Theory of Communication," *M.D. Computing*, 1997.

[22] W. Jin, A. K. Tung, J. Han, and W. Wang, "Ranking outliers using symmetric neighborhood relationship," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2006.

[23] M. M. Breuniq, H. P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *SIGMOD Record (ACM Special Interest Group on Management of Data)*, 2000.

[24] H. P. Kriegel, P. Kröger, E. Schubert, and A. Zimek, "LoOP: Local outlier probabilities," in *International Conference on Information and Knowledge Management, Proceedings*, 2009.

[25] H. Junninen, H. Niska, K. Tuppurainen, J. Ruuskanen, and M. Kolehmainen, "Methods for imputation of missing values in air quality data sets," *Atmospheric Environment*, vol. 38, pp. 2895–2907, 6 2004.

[26] Y. Yu, J. J. Q. Yu, V. O. K. Li, and J. C. K. Lam, "A novel interpolation-svt approach for recovering missing low-rank air quality data," *IEEE Access*, vol. 8, pp. 74 291–74 305, 2020.

[27] R. Salakhutdinov and A. Mnih, "Bayesian probabilistic matrix factorization using markov chain monte carlo," in *Proceedings of the 25th International Conference on Machine Learning*, ser. ICML '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 880–887. [Online]. Available: https://doi.org/10.1145/1390156.1390267

[28] L. Sun and X. Chen, "Bayesian temporal factorization for multidimensional time series prediction," 2019.

[29] Y. Yu, J. J. Q. Yu, V. O. K. Li, and J. C. K. Lam, "A novel interpolation-svt approach for recovering missing low-rank air quality data," *IEEE Access*, vol. 8, pp. 74 291–74 305, 2020.

[30] C. Viroli and G. J. McLachlan, "Deep Gaussian mixture models," 2017.

[31] E. Cuthill and J. McKee, "Reducing the bandwidth of sparse symmetric matrices," in *Proceedings of the 1969 24th National Conference, ACM 1969*, 1969.

[32] U. Von Luxburg, "A tutorial on spectral clustering," *Statistics and Computing*, 2007.

[33] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural Computation*, 2003.

[34] L. O. Mafteiu-Scai, "The Bandwidths of a Matrix. A Survey of Algorithms," *Annals of West University of Timisoara - Mathematics*, 2015.

[35] A. Azad, M. Jacquelin, A. Buluc, and E. G. Ng, "The Reverse Cuthill-McKee Algorithm in Distributed-Memory," in *Proceedings - 2017 IEEE 31st International Parallel and Distributed Processing Symposium, IPDPS 2017*, 2017.